

Final Project

Team Name: Simon Says What

A Multi-Player Hex-Memory Game on FPGA

ECE 6370 Advanced Digital Design

Dr. Yuhua Chen

Authors:

Ajay Doppalapudi

Zaki Mir

Tru Nguyen

Bryan Perez

May 12, 2026

Introduction

This final project implements a single-player hex-sequence memory game on the Terasic DE0-CV FPGA development board (Cyclone V 5CEBA4F23C7). The game challenges the player to memorize and reproduce a sequence of randomly generated hexadecimal digits within a 99-second round, with score accumulation weighted by a selectable difficulty level. The design extends the Lab 4 binary math game by adding a dual-credential authentication system, a multi-state game lifecycle FSM, a persistent per-player and global leaderboard backed by on-chip BRAM, and a unified state-driven HEX display renderer.

The project introduces several new architectural elements over Lab 4. The Access Controller is replaced by an Authentication module that now performs two-stage credential checking: a 4-digit Player ID followed by a 4-digit password. Both credentials are stored in dedicated 32-entry ROMs (PlayerID_ROM and Password_ROM) and are accessed by sub-FSMs ID_Auth / PW_Auth that scan the ROM table linearly. The Game Controller (gameControl) is expanded from 5 states to 7, adding an explicit ID_IN state between UNAUTHENTICATED and AUTHENTICATED, and replacing the GAME_OVER state with the END_GAME and LBOARD pair that handle the post-round score write and leaderboard view. A new one-hot output bus, display_mode[6:0], replaces the multiple discrete control signals previously routed to the HEX renderer; it drives LEDR[6:0] for phase indication and serves as the single phase input to DisplayCnt.

The gameplay loop is implemented inside a wrapper module called the MEGACOMPARATER, which is composed of three cooperating sub-modules: a Linear-Feedback Shift Register based Random Number Generator (RNG_Comparator with the lfsr_rng_rev1 core), a Comparator FSM that handles the round-by-round sequence display and player input capture, and a saturating accumulator (score_unit) that tracks the score with the difficulty multiplier applied on each correct match. The score is committed to BRAM at end-of-round through a 9-state controller (scoreRAM_Ctrl) wrapped together with the BRAM block (score_RAM) inside scoreRAM_top. A separate selector module, ScoreRAM_adapter, exposes a single 24-bit packed leaderboard entry to DisplayCnt and toggles between the per-player best and the all-time global best on the scroll button.

Reused modules from Lab 4 include twoDigitTimer (with the LFSRTimer-based one-second tick), digitTimer, countTo2, countTo3, oneSecTimerLFSR, and LFSRTimer. The button shaper from Lab 4 (sim_bs) is reworked into a polarity-flipped, registered-output variant called btn_shaper that emits an active-HIGH single-cycle pulse on the press edge; three instances are used in the new top level. The legacy Adder_4bit and Match_LED modules are removed since the gameplay is no longer additive matching.

System Architecture Design

The system is composed of the following Verilog modules: the top-level module Final_Project, the Button Shaper (btn_shaper, three instances), the Authentication wrapper (Authentication) containing the ID Authentication sub-module (ID_Auth) and the Password Authentication sub-module (PW_Auth), the ID checker (ID_check) and password checker (PW_check) helper FSMs, the two ROMs (PlayerID_ROM and Password_ROM), the Game Controller (gameControl), the Two Digit Timer (twoDigitTimer) which includes the LFSR-based one second timer (oneSecTimerLFSR) and its LFSRTimer / countTo3 / countTo2 sub-modules along with digitTimer instances, the MEGACOMPARATER wrapper (megaComparator) containing the Comparator FSM, the LFSR-based RNG (RNG_Comparator using lfsr_rng_rev1 and LFSR primitives), the saturating Score Unit (score_unit), the ScoreRAM wrapper (scoreRAM_top) which contains the ScoreRAM Controller (scoreRAM_Ctrl) and the BRAM block (score_RAM), the ScoreRAM Adapter (ScoreRAM_adapter), and the HEX renderer (DisplayCnt).

The top-level module instantiates the button shaper three times (KEY[1] for player action, KEY[2] for authentication digit entry and game start, KEY[3] for logout), the Authentication wrapper once, the gameControl once, the twoDigitTimer once, the megaComparator once, the scoreRAM_top once, the ScoreRAM_adapter once, and the DisplayCnt once. KEY[0] (reset) is wired directly to all modules without shaping.

Top-Level Block Diagram

Figure 1 shows the top-level block diagram of the Final Project. The diagram shows the data flow between Authentication, gameControl, the MEGACOMPARATER block, the RoundTimer (twoDigitTimer), ScoreRAM, and DisplayCnt. Authentication delivers the matched PlayerID (5-bit ROM index) and the 16-bit NumberID to ScoreRAM. gameControl drives every other module via control signals (enable, difficulty, write pulse, scroll pulse, soft reset). The MEGACOMPARATER produces the live score and the displayed random nibble, both consumed by ScoreRAM and DisplayCnt respectively.

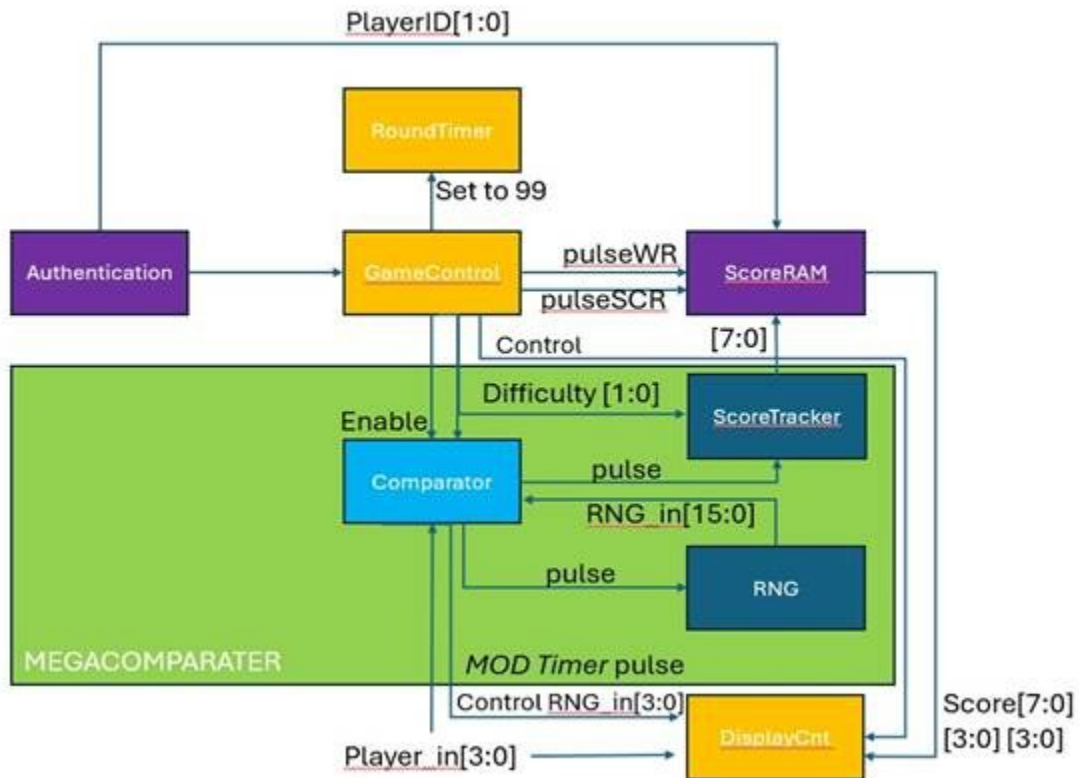


Figure 1 — Top-level System Block Diagram showing Authentication, gameControl, RoundTimer, MEGACOMPARATER (Comparator + ScoreTracker + RNG), ScoreRAM, and DisplayCnt.

Table 1: Top-Level Signal Definitions

Signal	Direction	Description
CLOCK_50	Input	50 MHz system clock; wired to every sequential block.
KEY[0]	Input	RESET — active-LOW, wired directly to all modules without shaping.
KEY[1]	Input	Player action button (active-LOW, shaped). Player nibble load during GAME_RUNNING; leaderboard scroll during LBOARD.
KEY[2]	Input	Authentication digit Enter (UNAUTHED / ID_IN); Game Start (WAIT_FOR_START); Round Restart (LBOARD). Shaped, active-LOW raw → active-HIGH single-cycle pulse.
KEY[3]	Input	Logout button — shaped, active-HIGH pulse to gameControl in WAIT_FOR_START and LBOARD only.
SW[3:0]	Input	Player nibble during GAME_RUNNING.
SW[5:4]	Input	Difficulty select: 00 = none, 01 = easy (20 s), 10 = medium (15 s), 11 = hard (10 s).
SW[9:6]	Input	ID / password digit input during authentication.
HEX0[6:0]	Output	Random nibble during gameplay; least-significant score nibble in LBOARD.
HEX1[6:0]	Output	Score and leaderboard digit.
HEX2[6:0]– HEX3[6:0]	Output	Leaderboard NumberID digits.
HEX4[6:0]	Output	Timer ones digit (second from left).
HEX5[6:0]	Output	Timer tens digit (far left).
LEDR[6:0]	Output	One-hot phase indicator from gameControl: [0] UNAUTHED, [1] ID_IN, [2] AUTHED, [3] WAIT_FOR_START, [4] GAME_RUNNING, [5] END_GAME, [6] LBOARD.
LEDR[9:7]	Output	Reserved — tied LOW (or used for debug probes during integration).

Button Shaper (btn_shaper)

The Button Shaper is a polarity-flipped, registered-output rework of the Lab 2 / Lab 4 `sim_bs` module. It produces an active-HIGH single-cycle pulse on the falling edge of an active-LOW raw button input, then re-arms only after the player releases the button. The output polarity inversion was required because the new top-level consumers (`Authentication.B_in_shaped` and `Comparator.player_load_in`) both expect active-HIGH pulses, whereas the Lab 4 design used active-LOW shaped pulses.

The FSM has three states: IDLE (output LOW, waiting for press), PULSE (output HIGH for exactly one clock cycle), and WAIT (output LOW, waiting for release). The output is registered so it is glitch-free for downstream synchronous consumers.

Three instances are used in the top level: one for `KEY[1]` (player action), one for `KEY[2]` (authentication digit enter / game start / round restart), and one for `KEY[3]` (logout). `KEY[0]` (reset) passes through raw and is not shaped.

Authentication Block

The Authentication block performs a two-stage credential check: a 4-digit Player ID match, followed by a 4-digit password match for the matched ID. The block is composed of a top-level Authentication wrapper that instantiates the ID Authentication sub-module (`ID_Auth`) and the Password Authentication sub-module (`PW_Auth`). The two sub-modules expose their matched outputs to the wrapper, which forwards them to the rest of the system: `ID_passed` (HIGH after a valid ID), `LoggedIn` (HIGH after a valid password), the 5-bit `PlayerID` (ROM index of the matched account), and the 16-bit `NumberID` (the original ID hex value, used downstream for leaderboard display).

ID Authentication (`ID_Auth` / `ID_check`)

`ID_Auth` manages the four-digit ID entry. The FSM has eight states: `INIT`, `DIGIT1`, `DIGIT2`, `DIGIT3`, `DIGIT4`, `FETCHROM`, `ROMWAIT1`, `ROMWAIT2`, `CATCHROM`, `COMPARE`, `PWSTATUS`, and `IDPASSED`. The four `DIGITn` states capture each successive 4-bit switch value on `KEY[2]` press, packing the digits into a 16-bit `EnterID_reg` in MSB-first order. After all four digits are captured, the FSM walks the 32-entry `PlayerID_ROM` linearly, comparing the captured value against each stored ID; on a match, it latches the 5-bit ROM address as the matched `PlayerID` and the 16-bit ROM data as the `NumberID`, then transitions to `IDPASSED` and raises `ID_passed`.

`ID_check` is the comparator helper that holds the ROM read pipeline for the registered-output BRAM. Because the ROM has a 1-clock latency between address-set and data-valid, the FSM inserts `FETCHROM` → `ROMWAIT1` → `ROMWAIT2` → `CATCHROM` as a three-cycle pipeline padding before the `COMPARE` step. This pattern is identical between `ID_Auth` and `PW_Auth`.

If the FSM walks all 32 ROM addresses without a match, the read returns `16'hFFFF` (the sentinel value loaded into unused slots), and the FSM increments a fail counter; after four failures, the system can either lock out or reset depending on the design choice. The standard configuration resets back to `INIT` and lets the user retry.

Password Authentication (PW_Auth / PW_check)

PW_Auth mirrors ID_Auth's structure but operates against Password_ROM. The four password digits are captured in DIGIT1–DIGIT4, packed MSB-first into EnterPW_reg, and then linearly compared against the 32-entry Password_ROM. The linear search is intentional: it allows the credential pairing (ID at addr N, password at addr N) to be set up by simply storing pairs in matching addresses across the two ROMs, but it does not actually require the password to be at the same index as the matched ID — any password in the ROM will pass. This simplifies the FSM and avoids passing the matched PlayerID between sub-FSMs at the cost of weaker credential binding.

On a successful password match, PW_Auth raises LoggedIn and latches the PlayerID_from_ID_Auth into PlayerID_to_scoreRAM, along with NumberID_from_ID_Auth. The FSM then enters a PWPASSED hold state until gameControl asserts Logout_Req, at which point it returns to INIT and clears LoggedIn.

ROM Storage

Each ROM is a Quartus-generated 32 x 16-bit single-port BRAM with registered output. The PlayerID_ROM stores the four hex-digit player IDs at addresses 0–3, with remaining slots filled with 16'hFFFF as the end-of-list sentinel. The Password_ROM stores the matching passwords in the same address positions. Both ROMs use a .hex initialization file placed in the project synthesis directory.

ID/Password Pair Storage

ROM Index	PlayerID	Password
0	0x1234	0x5810
1	0x5678	0x4525
2	0xABCD	0x3998
3	0xCDEF	0x1032
4–31	0xFFFF (end-of-list)	0xFFFF (end-of-list)

Game Controller (gameControl)

The Game Controller is the master FSM that orchestrates the round lifecycle. It is expanded from the 5-state Lab 4 design to 7 states to accommodate the dual-stage authentication, the post-round write-and-display sequence, and the leaderboard view.

State Graph

State	Encoding	display_mode (LEDR)	Purpose
UNAUTHENTICATED	4'd1	7'b000_0001 (LEDR[0])	Waiting for ID match. Auth FSM in ID entry.
ID_IN	4'd7	7'b000_0010 (LEDR[1])	ID matched; awaiting password. Auth FSM in PW entry.
AUTHENTICATED	4'd2	7'b000_0100 (LEDR[2])	PW passed; pulse timer_reconfig HIGH for one cycle to load 99. Single-cycle pass-through.
WAIT_FOR_START	4'd3	7'b000_1000 (LEDR[3])	Difficulty selection; awaiting KEY[2] press to start the round, or KEY[3] press to log out.
GAME_RUNNING	4'd4	7'b001_0000 (LEDR[4])	Gameplay active. timer_enable HIGH, enable_mega_compar HIGH.
END_GAME	4'd5	7'b010_0000 (LEDR[5])	1-second hold post-round. pulse_w_ram fires for one cycle on entry to commit the round score to BRAM.
LBOARD	4'd6	7'b100_0000 (LEDR[6])	Leaderboard view. KEY[1] scrolls (toggle MyBest/GlobalBest), KEY[2] restarts the round, KEY[3] logs out.

Key Transitions

The transitions are: UNAUTHENTICATED → ID_IN on ID_passed HIGH; ID_IN → AUTHENTICATED on verified (LoggedIn) HIGH; AUTHENTICATED → WAIT_FOR_START unconditionally on the next clock (single-cycle); WAIT_FOR_START → GAME_RUNNING on game_start_key HIGH and difficulty_in non-zero; GAME_RUNNING → END_GAME on timer_done HIGH (pulse_w_ram raised in the same cycle); END_GAME → LBOARD when the internal 26-bit hold counter reaches END_GAME_HOLD (49,999,999 cycles at 50 MHz = 1.0 s); LBOARD → WAIT_FOR_START on game_start_key HIGH (round restart with previously-selected difficulty); WAIT_FOR_START → UNAUTHENTICATED and LBOARD → UNAUTHENTICATED on id_log_inout HIGH (logout pulse from KEY[3]), with logout_req asserted to reset the Authentication FSM.

display_mode Output

The 7-bit one-hot display_mode bus is the single source of truth for the system's current phase. It is wired directly to LEDR[6:0] for visual indication and also fed to DisplayCnt as its phase selector input. This replaces the multiple discrete display-control signals that the Lab 4 design routed to DisplayCnt and the LEDR pins.

END_GAME Hold Counter

On entry to END_GAME the gameControl raises pulse_w_ram HIGH for exactly one clock cycle to trigger ScoreRAM's write FSM. It then holds in END_GAME for one full second (50,000,000 clocks at 50 MHz) using an internal 26-bit counter. This gives ScoreRAM the time to complete its 9-state write sequence (FETCHRAM → RAMWAIT1 → RAMWAIT2 → CATCHRAM → COMPARE → WRITETORAM → SCOREOUTPUT → WAITFORSCORE) well before LBOARD opens and starts reading from the same address bus.

Two Digit Timer (twoDigitTimer)

The Two Digit Timer is reused unmodified from Lab 4. It combines the LFSR-based one second timer (oneSecTimerLFSR) with two digit timer instances (ones and tens) to create a full 99-second countdown timer. The one second timer produces a 1-second pulse that drives the ones digit borrowDwn input. When the ones digit wraps from 0 to 9, it sends a borrowUp pulse to the tens digit. The timer_done output goes high when both digits reach 00 and timer_enable is still active. timer_reconfig is a one-shot pulse from gameControl that reloads both digits to 99.

Sub-modules

oneSecTimerLFSR chains LFSRTimer (~1 ms pulse) → countTo3 (~100 ms pulse from 100 of the 1 ms ticks) → countTo2 (1 s pulse from 10 of the 100 ms ticks). LFSRTimer is a 16-bit

Linear Feedback Shift Register with taps at bits 15, 5, 4, and 2 (polynomial $x^{16} + x^5 + x^4 + x^2 + 1$) and a terminal value of 16'h1C51, which matches a 1 ms timeout at 50 MHz. The digitTimer module implements a single BCD countdown digit with borrow-chain logic between adjacent digits.

MEGACOMPARATER (megaComparator)

The MEGACOMPARATER is the gameplay engine. It is a wrapper module containing three cooperating sub-modules: the Random Number Generator (RNG_Comparator), the Comparator FSM, and the saturating Score Unit (score_unit). Together they produce the displayed random nibble during gameplay and the live 8-bit score that ScoreRAM samples at end-of-round.

Comparator

The Comparator is a state machine that drives the per-window gameplay loop. When gameControl asserts enable_mega_compar HIGH (during GAME_RUNNING), the Comparator generates a new 4-bit random nibble (via a hold-and-capture pull from the free-running RNG), displays it on HEX0 for the difficulty-paced window (20 s easy, 15 s medium, 10 s hard), and then opens a window for the player to enter a matching value on SW[3:0] and confirm with KEY[1] (player_load_in shaped). On a successful match, the Comparator emits a score_pulse to the score_unit, which adds the difficulty multiplier to the running score. The Comparator then loops back to display the next random nibble, repeating until enable_mega_compar drops on round end.

Random Number Generator (RNG_Comparator)

The RNG is implemented as a 16-bit Linear Feedback Shift Register (lfsr_rng_rev1) using the LFSR primitive. The LFSR runs free-running at the system clock; the Comparator captures a 4-bit slice on demand and holds it stable for the duration of the display window. The maximal-length 16-bit polynomial provides a 65,535-state pseudo-random sequence with effectively uniform distribution across the 16 hex values when sampled at human-input timescales.

Score Unit (score_unit)

The score_unit is an 8-bit saturating accumulator. On each score_pulse it adds the 2-bit multiplier (1, 2, or 4 — corresponding to easy / medium / hard difficulty levels) to the current score, saturating at 8'd255 to prevent rollover. A soft_rst input from gameControl clears the score to zero at the start of each round (in WAIT_FOR_START and earlier phases) and preserves it during gameplay, END_GAME, and LBOARD. The live score output is wired both to ScoreRAM (sampled at the end-of-round write pulse) and to DisplayCnt (for in-round score display).

Difficulty Progression

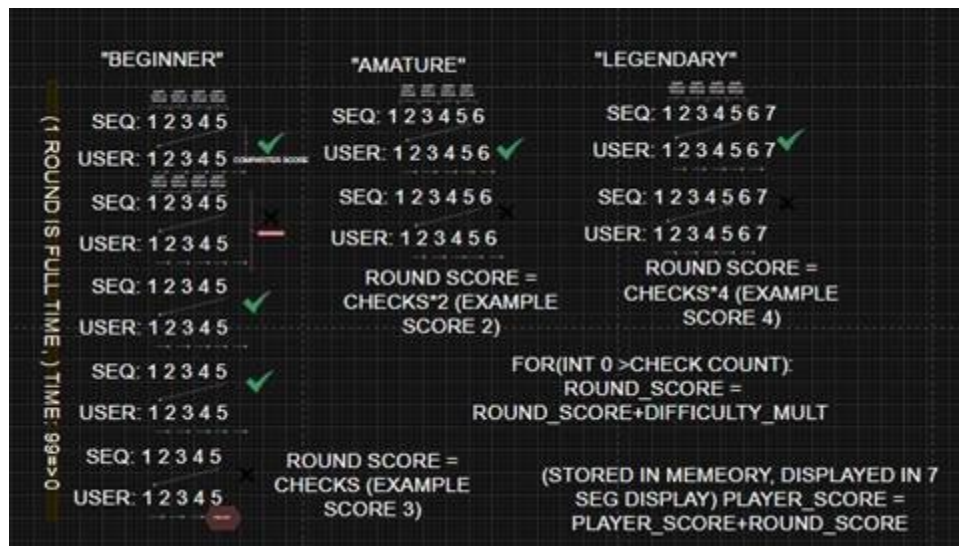


Figure 2 — Difficulty progression. BEGINNER (easy) shows 5 nibbles per round with a $1\times$ score multiplier; AMATURE (medium) shows 6 nibbles per round with a $2\times$ multiplier; LEGENDARY (hard) shows 7 nibbles per round with a $4\times$ multiplier. Round score = correct matches $\times$ multiplier.

ScoreRAM Block

The ScoreRAM block stores per-player and global high scores in on-chip BRAM and exposes two 24-bit packed outputs for the leaderboard display. The block consists of the scoreRAM_top wrapper, the scoreRAM_Ctrl FSM, the BRAM primitive (score_RAM), and the ScoreRAM_adapter selector that bridges the two parallel ScoreRAM outputs to DisplayCnt's single 24-bit lboard entry input.

scoreRAM Ctrl

scoreRAM_Ctrl is a 9-state FSM: RAMINIT, WAITFORSCORE, FETCHRAM, RAMWAIT1, RAMWAIT2, CATCHRAM, COMPARE, WRITETORAM, and SCOREOUTPUT. On reset the FSM enters RAMINIT and walks the 32-entry BRAM clearing every address to 0x000000 (24-bit packed value of {NumberID, score}). It then enters WAITFORSCORE and remains there until gameControl asserts pulse_w_ram HIGH for one clock; the FSM latches LatestScore_reg from the live megaComparator score output, sets RAMAddr to the matched PlayerID from Authentication, and steps through FETCHRAM → RAMWAIT1 → RAMWAIT2 → CATCHRAM to handle the BRAM's registered-output read pipeline. In CATCHRAM it captures the stored RAM value into RAMdata_reg and refreshes MyBest_reg from the current player's BRAM slot (preventing leakage from a previous player). In COMPARE, if the new score exceeds the stored value, the FSM writes the new {NumberID, score} packed value back to the same address and updates MyBest_reg. In SCOREOUTPUT, it also updates the global best if the new score

exceeds GlobalBest_reg, then commits both packed display outputs (MyBest_to_Dispcnt and GlobalBest_to_Dispcnt) in the same cycle so they are valid immediately when LBOARD opens. The FSM returns to WAITFORSCORE for the next pulse.

score_RAM

score_RAM is a Quartus IP wizard-generated 32 x 24-bit single-port BRAM with registered output. The 24-bit width carries the packed {NumberID[15:0], score[7:0]} pair per slot. The address width is 5 bits to address all 32 slots. The registered output adds one cycle of read latency, accommodated by the 3-cycle RAMWAIT pipeline in scoreRAM_Ctrl.

ScoreRAM_adapter

The ScoreRAM_adapter is a small selector that sits between scoreRAM_top (which exposes two parallel 24-bit outputs) and DisplayCnt (which accepts a single 24-bit lboard_entry input). It maintains a 1-bit view_sel register: 0 selects MyBest, 1 selects GlobalBest. pulse_w_ram from gameControl resets view_sel to 0 (opening LBOARD at MyBest after each round). pulse_scl_ram from gameControl (one-cycle pulse on KEY[1] press during LBOARD) flips view_sel. The output is registered so it is glitch-free into DisplayCnt's already-registered HEX path.

Display Counter (DisplayCnt)

DisplayCnt is the unified HEX renderer. It selects one of seven HEX layouts based on the 7-bit one-hot display_mode input from gameControl. Each layout drives all six HEX displays in a coordinated pattern appropriate to the current phase. Internally, DisplayCnt is a multiplexer of pre-decoded 7-segment patterns; per-state the input ones_count / tens_count (from twoDigitTimer), DisplayRandom (from megaComparator), and lboard_entry (from ScoreRAM_adapter) are selected onto the HEX outputs.

Per-State HEX Layout

display_mode state	HEX5	HEX4	HEX3	HEX2	HEX1	HEX0
UNAUTHED / ID_IN	blank	blank	blank	blank	blank	echo digit_auth
AUTHED (1-cycle)	9	9	blank	blank	blank	blank
WAIT_FOR_START	9	9	blank	blank	blank	blank
GAME_RUNNING	tens_count	ones_count	blank	score H	score L	DisplayRandom
END_GAME	0	0	blank	blank	blank	blank
LBOARD	ID[15:12]	ID[11:8]	ID[7:4]	ID[3:0]	score[7:4]	score[3:0]

FPGA Board Testing Results

The design was synthesized in Quartus Prime 25.1std.0 Lite Edition targeting the Cyclone V 5CEBA4F23C7 device and downloaded to the DE0-CV board. Pin assignments follow the standard Terasic DE0-CV golden-top reference: CLOCK_50 on PIN_M9, KEY[3:0] on PIN_M6/M7/W9/U7, SW[9:0] on the U/V/T/AA/AB column, LEDR[9:0] on AA/W/Y/N/U/L row pins, and the six HEX displays on their respective banks. The full pin map is documented in Final_Project.qsf.

Authentication Test

The four stored credential pairs were tested in turn: (1234, 5810), (5678, 4525), (ABCD, 3998), and (CDEF, 1032). Each ID/password pair successfully advanced the system from LEDR[0] (UNAUTHED) through LEDR[1] (ID_IN) to LEDR[3] (WAIT_FOR_START), with the brief LEDR[2] flash on the AUTHENTICATED single-cycle transition not visible to the naked eye. Incorrect digits at any stage held the FSM at the current digit position and allowed retry without resetting earlier-entered digits.

Logout Path

KEY[3] was tested as a logout from both WAIT_FOR_START and LBOARD. The shaped single-cycle pulse correctly returned the system to UNAUTHED (LEDR[0]) by asserting logout_req back to the Authentication FSM, which cleared LoggedIn and returned to INIT. The dual-role KEY[2] (auth digit entry / game start / restart) was verified to be unambiguous across all phases because the consumers are mutually exclusive by gameControl state.

Gameplay and Scoring

The 99-second timer counted down correctly from 99 to 00 on HEX5–HEX4 across all three difficulty levels. The Comparator displayed a fresh random nibble on HEX0 at the difficulty-paced interval, and successful matches incremented the score visibly during gameplay. The end-of-round write pulse correctly triggered the scoreRAM_Ctrl FSM to commit the round score to BRAM, with the leaderboard opening at the player's own record immediately on entry to LBOARD.

Leaderboard

KEY[1] (leaderboard scroll) correctly toggled between MyBest and GlobalBest in LBOARD, with the ScoreRAM_adapter flipping view_sel on each press. KEY[2] (round restart) returned the system to WAIT_FOR_START with the previously latched difficulty preserved. KEY[3] (logout) returned to UNAUTHED as expected.

Video

[video_FinaProject_SimonSayWhat](#)

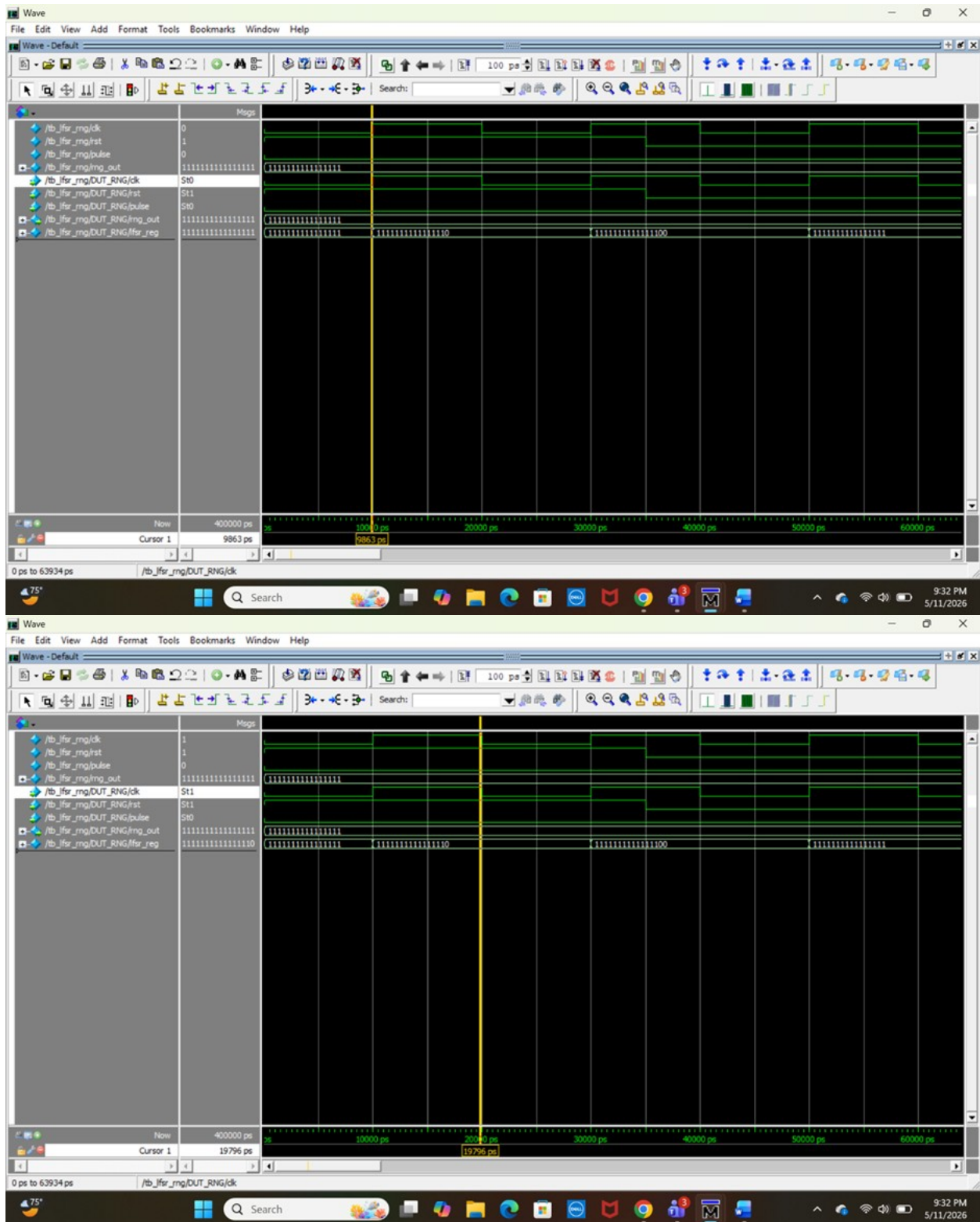
Conclusion

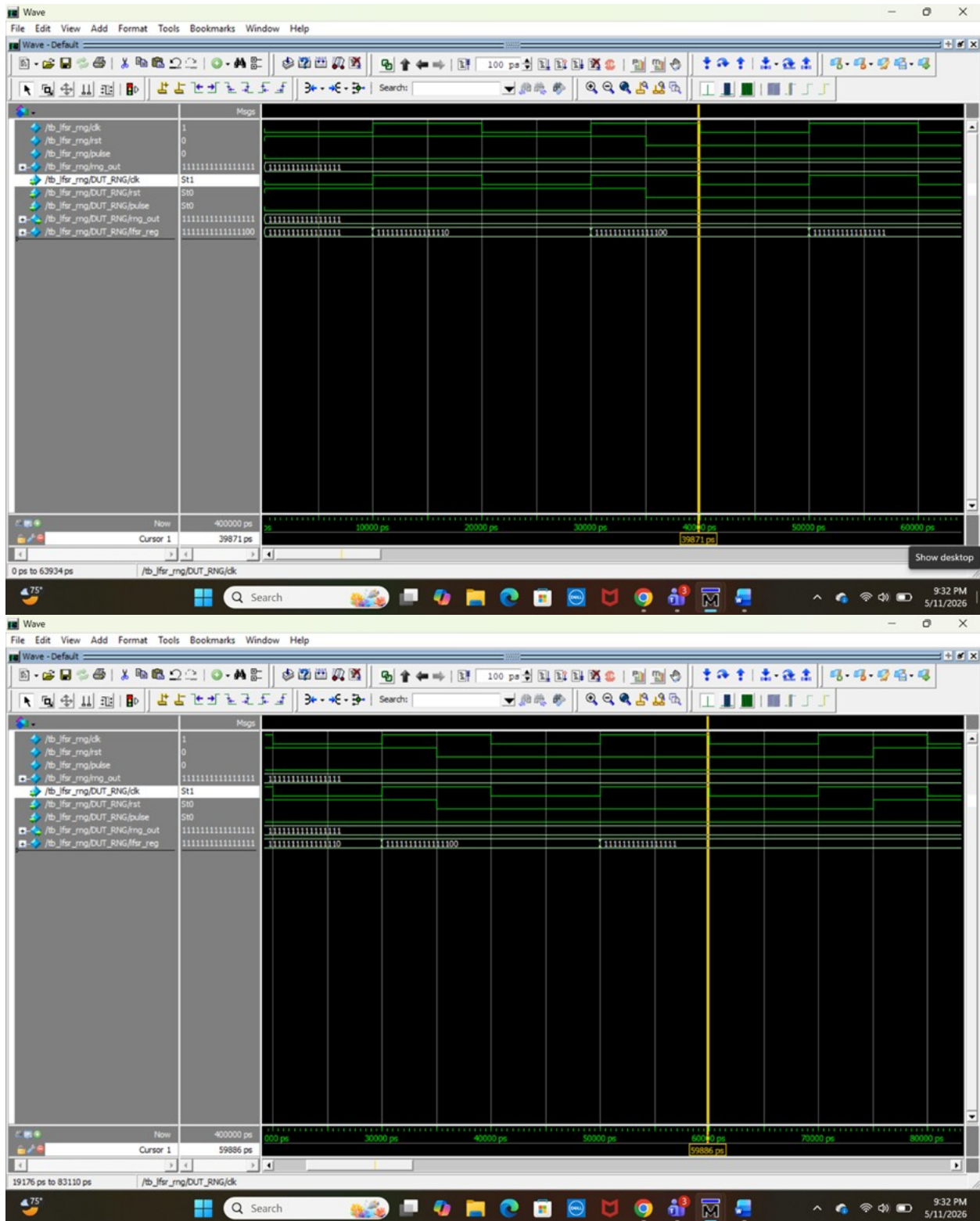
The Final Project successfully integrates a multi-stage authentication system, a 7-state game lifecycle controller, an LFSR-driven hex-sequence gameplay engine, and a BRAM-backed dual-view leaderboard onto the DE0-CV FPGA board. The architecture builds incrementally on Lab 4: the LFSR-based timer, button shaper, and digit-timer primitives are carried over without modification, while the Access Controller is replaced by the Authentication wrapper with paired ID and password ROMs, and the binary-math Adder + Match_LED pair is replaced by the MEGACOMPARATER engine.

The design demonstrates several practical hardware techniques: ROM-based credential storage with a registered-output read pipeline, BRAM-backed high-score persistence with a 9-state write FSM, one-hot phase indication that doubles as both a UI signal and a HEX-renderer selector, and active-HIGH shaped button pulses for state-mutually-exclusive dual-role inputs (KEY[2] for both auth digit entry and game start; KEY[3] for logout-only after the polarity fix during integration). Edge cases were resolved during board bring-up: the held-key timing issue during KEY[3] in the original dual-role auth-and-logout design was eliminated by moving auth digit entry onto KEY[2] and making KEY[3] logout-only with an active-HIGH shaped pulse.

Future extensions could include a second player slot driven by SW[3:0] or external GPIO, a more sophisticated RNG using physical entropy harvested from button-press intervals, and per-difficulty leaderboard slots so the global best is computed only against comparable difficulty entries.

Simulation:

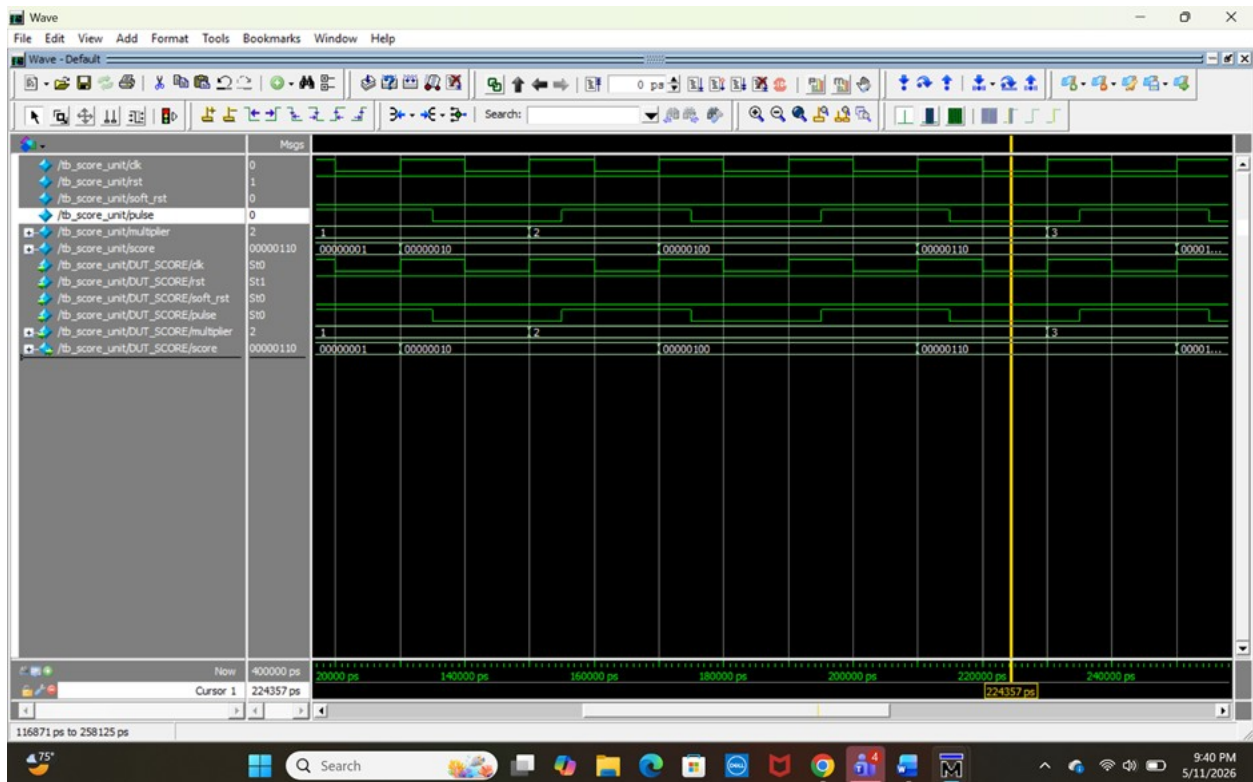


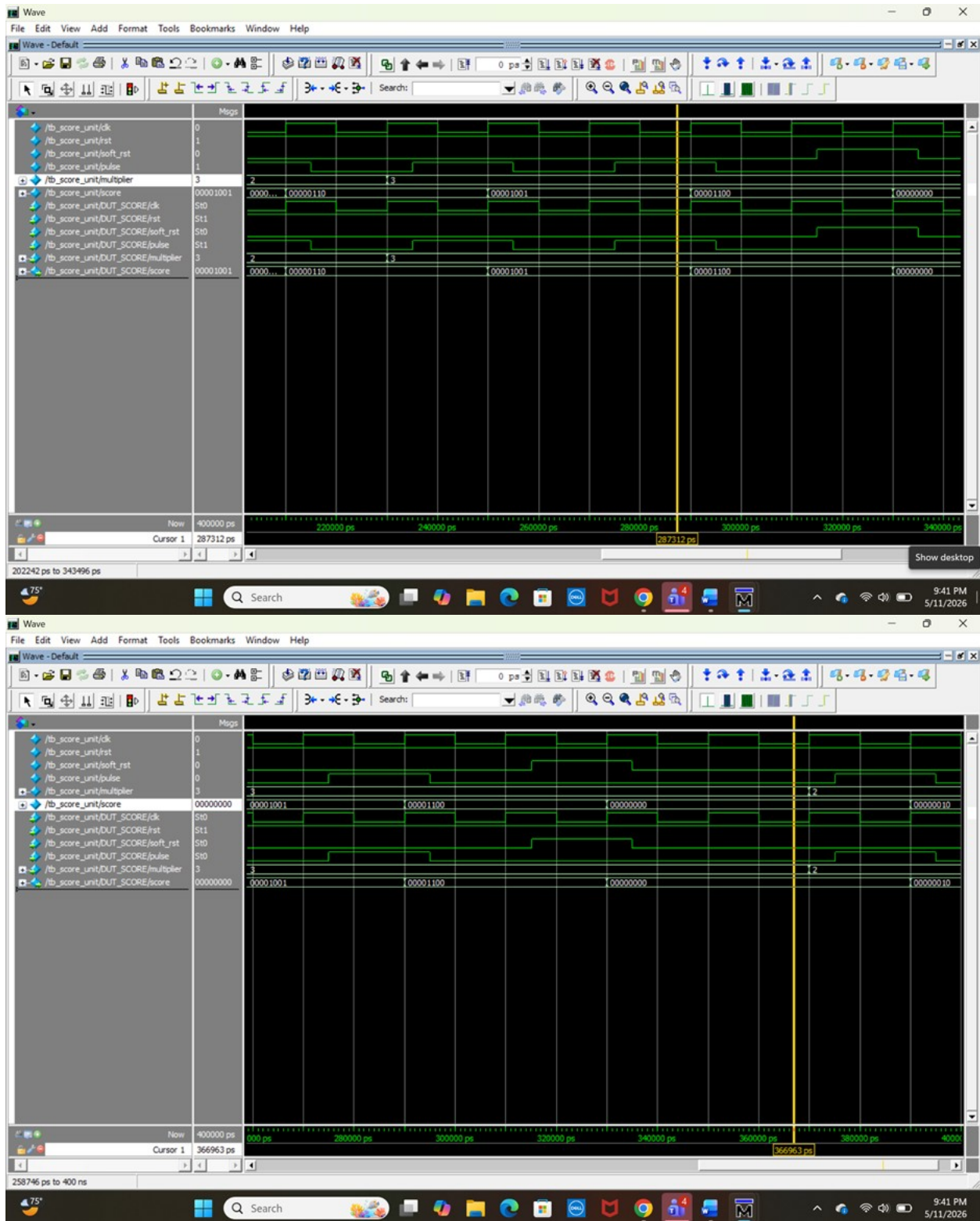


The simulation waveform confirms that upon applying the active-low reset, the internal `lfsr_reg` and `rng_out` are both correctly initialized to the seed value 1111111111111111 (16'hFFFF). Once reset is released, the LFSR register shifts continuously on every rising clock edge, producing a

new pseudo-random state each cycle as seen by the transitions through 111111111111110, 111111111111100, and so on. The rng_out output correctly holds its last captured value between pulse events, confirming proper sample-and-hold functionality, and the second reset application mid-simulation also restores the register to the seed value as expected.

Simulation results of Score tracker



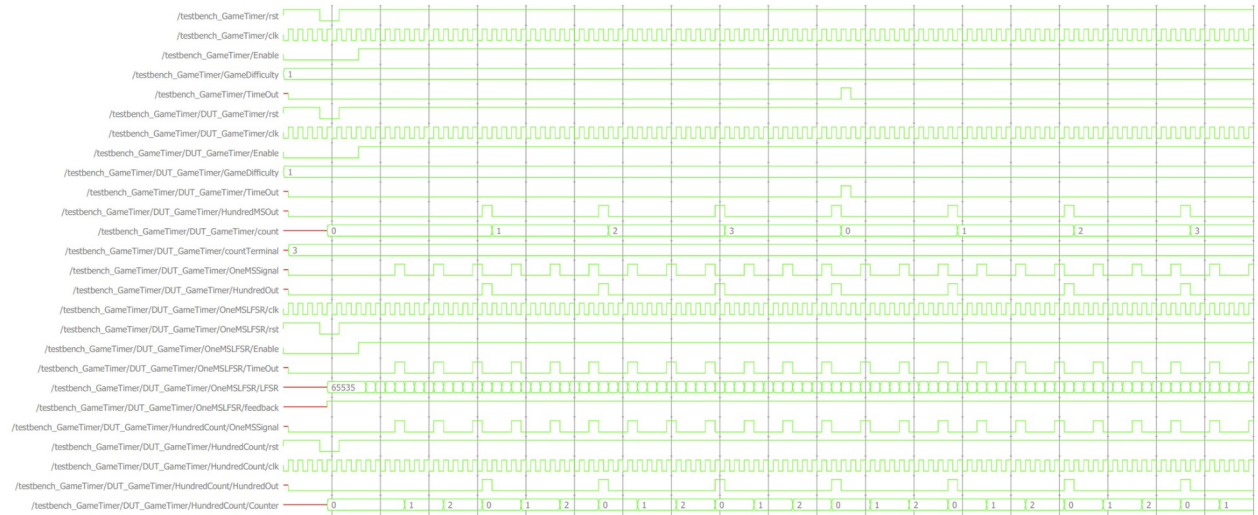


The simulation waveform verifies that the score increments correctly according to the selected difficulty multiplier advancing by 1 in easy mode, 2 in medium mode, and 3 in hard mode with each increment occurring one clock cycle after the rising edge of the pulse signal, confirming

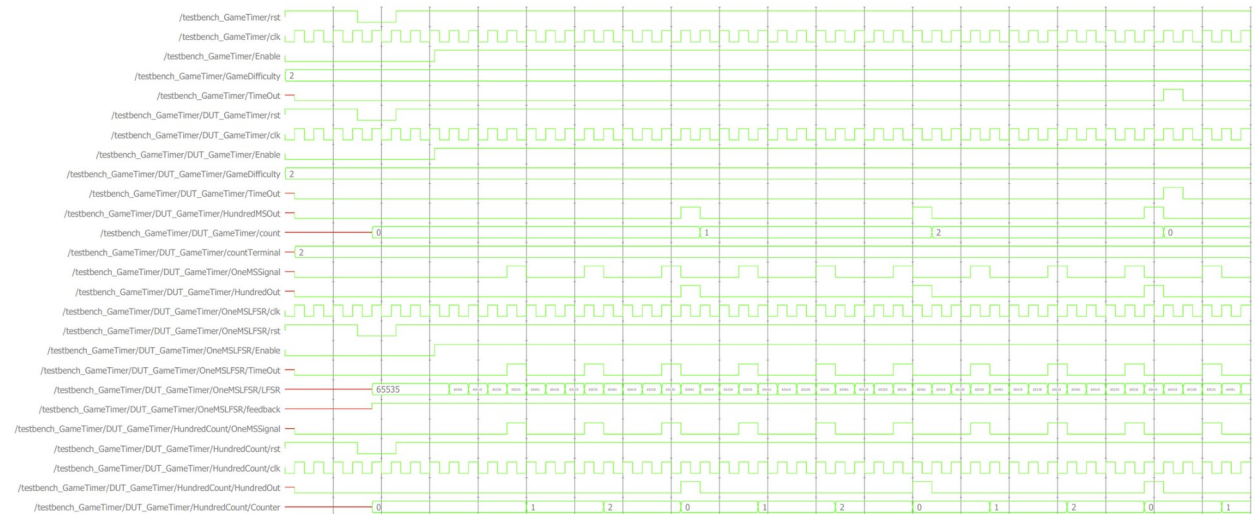
synchronous operation. The soft reset signal is observed to immediately clear the score to zero on the next clock edge without affecting the multiplier setting, demonstrating correct intra-round score clearing behavior. Following the soft reset, scoring resumes from zero correctly, and the subsequent assertion of the active-low hard reset also clears the score as expected, confirming that all priority levels in the reset hierarchy operate correctly.

GameTimer: Reconfigurable Timer

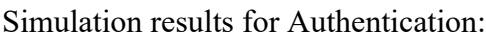
Level 1

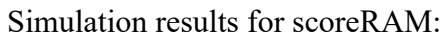


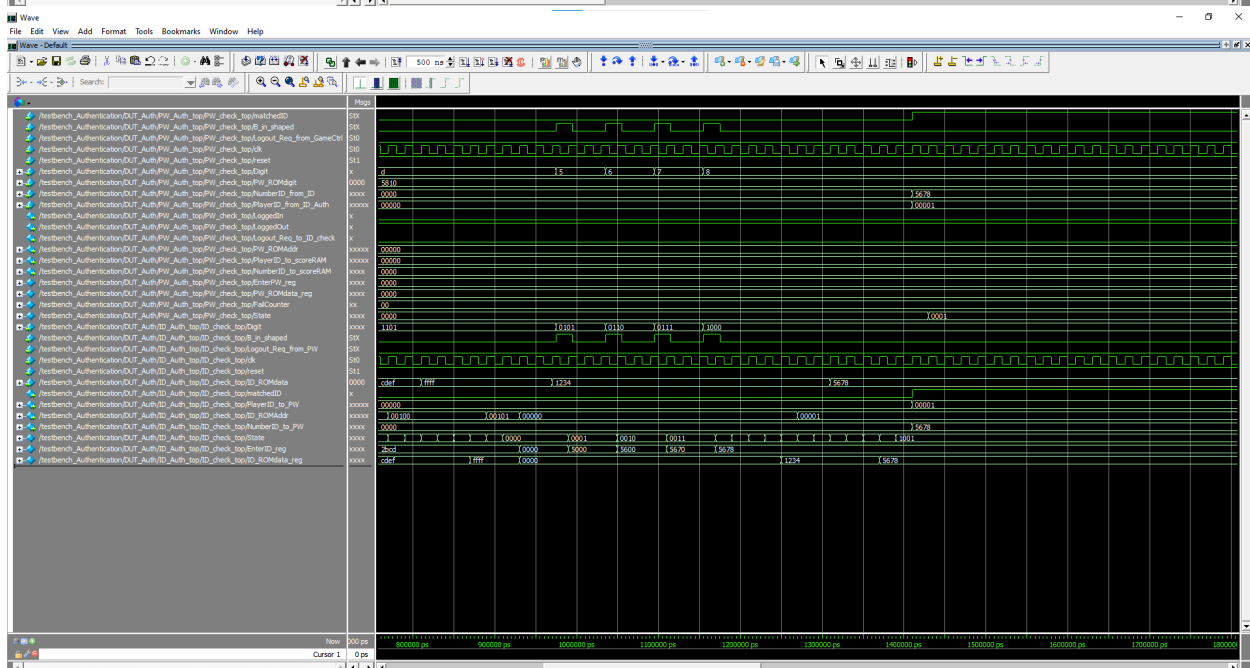
Level 2



Level 3







Top Module

```
Final_Project.v
1 // Final Project Top-Level Module
2 /*
3 =====
4 Course:      ECE 6370 Advanced Digital Design
5 Author:      Zaki Mir
6 PeopleSoft ID: 1675819
7 Project:     Final Project - "Hex Sequence"
8 File:        Hex_Sequence_top.v
9 Target:      Terasic DE0-CV (Cyclone V 5CEBA4F23C7N)
10 Rev:         3 -- Lab 4 style header + named binding
11 =====
12
13 Project Description:
14   Single-player hex-sequence memory game (Simon-Says variant) on
15   the DE0-CV. The player logs in with a 4-digit ID and 4-digit
16   password (both stored in ROM, walked by sequential FSMs). After
17   picking a difficulty (easy/medium/hard), the player has 99
18   seconds to score as many rounds as possible. Each round the
19   RNG shows four scrambled nibbles on HEX0 at a difficulty-paced
20   interval; the player then re-enters those four nibbles on
21   SW[3:0] using KEY[1]. Successful matches add to a difficulty-
22   weighted score. At round end, a per-player and global high
23   score are written to BRAM. The leaderboard view lets the
24   player toggle between their own best and the all-time best.
25
26
27 Inputs:
28   CLOCK_50      -- 50 MHz system clock                      (same as Lab 4)
29   KEY[0]         -- RESET, active-LOW                        (same as Lab 4)
30   KEY[1]         -- game-action button (shaped):              (modified from Lab 4)
31                   -- player digit entry during PLAYING
32                   -- leaderboard scroll during LBOARD
33   KEY[2]         -- game start / round restart (shaped)      (modified from Lab 4)
34   KEY[3]         -- dual-role button:                          (new)
35                   -- shaped: ID/PW digit entry during auth
36                   -- raw active-LOW: logout from WAIT_FOR_START
37                   or LBOARD
38   SW[3:0]        -- player nibble input during PLAYING        (same as Lab 4)
39   SW[5:4]        -- difficulty select (00=none,               (new)
40                   01=easy/20s, 10=med/15s, 11=hard/10s)
41   SW[9:6]        -- ID / password digit entry                (same as Lab 4)
42
43 Outputs:
44   HEX5..HEX0    [6:0] -- six 7-seg displays, layout depends on phase (modified from Lab 4)
45                   (see DisplayCnt.v for state-by-state HEX map;
46                   summary: blank during auth, "99" in PRE_GAME,
47                   countdown+RNG in PLAYING, "00" in END_GAME,
48                   ID+score in LBOARD)
49   LEDR[6:0]     -- one-hot phase indicator from gameControl: (new)
50                   LEDR[0]=UNAUTHED, LEDR[1]=ID_IN,
51                   LEDR[2]=AUTHED,   LEDR[3]=PRE_GAME,
52                   LEDR[4]=PLAYING,  LEDR[5]=END_GAME,
53                   LEDR[6]=LBOARD
54   LEDR[9:7]     -- tied LOW (unused)                          (modified from Lab 4)
55
```

```

55
56 Module Instances:
57   btn_shaper x 3      -- shape KEY[1], KEY[2], KEY[3] into single-
58                       -- cycle HIGH pulses on press
59                       -- (KEY[0] passes through raw as reset)
60   Authentication      -- ID + PW match against PlayerID_ROM /
61                       -- Password_ROM; emits LoggedIn, ID_passed,
62                       -- PlayerID, NumberID
63   gameControl          -- master FSM, 7 states, one-hot display_mode
64                       -- output drives LEDR and DisplayCnt
65   twoDigitTimer        -- 99 -> 00 countdown using 1 ms LFSR tick +
66                       -- 100 ms counter; emits ones/tens/done
67   megaComparator       -- gameplay engine wrapping Comparator (RNG,
68                       -- nibble slicing, player match) +
69                       -- score_unit_rev1 (saturating accumulator)
70   scoreRAM_top         -- 32 x 24-bit BRAM + 9-state FSM; per-player
71                       -- high score plus global best
72   ScoreRAM_adapter     -- toggles between MyBest and GlobalBest on
73                       -- scroll pulse; opens at MyBest on round end
74   DisplayCnt          -- HEX renderer; picks one of 7 layouts based
75                       -- on display_mode
76
77 KEY[3] dual-role rationale:
78 In UNAUTH and ID_IN, Authentication reads B_in_shaped to advance
79 digit entry; gameControl in those states does NOT read
80 id_log_inout. In WAIT_FOR_START and LBOARD, gameControl reads
81 id_log_inout for logout; Authentication is in PWPASSED in those
82 states and does NOT react to B_in_shaped. The two consumers
83 are mutually exclusive by state, so a single press of KEY[3]
84 is unambiguous in every gameControl state.
85
86 KEY[1] dual-role rationale:
87 player_load_in is read by Comparator only in CatchPlayerInput
88 (active when gameControl is in GAME_RUNNING). game_control_key
89 is read by gameControl only in LBOARD. The two states are
90 mutually exclusive.
91
92 All instantiations use named-port binding to prevent
93 positional-mismatch errors during integration.
94 -----
95 + /

```

```

94  -----
95  */
96  module Final_Project(CLOCK_50, KEY, SW, LEDR, HEX0, HEX1, HEX2, HEX3, HEX4, HEX5);
97      input      CLOCK_50;          // 50 MHz system clock
98      input  [3:0] KEY;              // KEY[0]=RESET, [1]=game action, [2]=start, [3]=auth/logout
99      input  [9:0] SW;              // SW[3:0]=player, [5:4]=difficulty, [9:6]=auth digit
100
101      output [9:0] LEDR;             // LEDR[6:0]=display_mode (one-hot phase), [9:7] tied LOW
102      output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5; // driven by DisplayCnt per state
103
104      // =====
105      // Wire declarations
106      // =====
107
108      // shaped button outputs
109      wire key1_shaped, key2_shaped, key3_shaped;
110
111      // switch-derived signals
112      wire [3:0] digit_auth  = SW[9:6]; // ID/PW digit (Auth)
113      wire [3:0] digit_play  = SW[3:0]; // game nibble (Comparator)
114      wire [1:0] difficulty  = SW[5:4];
115      //wire      id_log_inout = KEY[3]; // raw active-LOW logout to gameControl
116
117      // from Authentication
118      wire      LoggedIn, LoggedOut, ID_passed;
119      wire [4:0] PlayerID_to_scoreRAM;
120      wire [15:0] NumberID_to_scoreRAM;
121
122      // from gameControl
123      wire      timer_enable, timer_reconfig;
124      wire      enable_mega_compar;
125      wire      pulse_w_ram, pulse_scl_ram;
126      wire [1:0] difficulty_out;
127      wire      logout_req;
128      wire      score_soft_rst;
129      wire [6:0] display_mode;
130
131      // from twoDigitTimer
132      wire [3:0] ones_count, tens_count;
133      wire      timer_done;
134

```

```

130
131 // from twoDigitTimer
132 wire [3:0] ones_count, tens_count;
133 wire timer_done;
134
135 // from megaComparator
136 wire [7:0] score;
137 wire [3:0] display_random;
138
139 // from scoreRAM_top
140 wire [23:0] my_best, global_best;
141
142 // from ScoreRAM_adapter
143 wire [23:0] lboard_entry;
144
145
146 // =====
147 // Module signature reference (for quick scan of port orders):
148 //
149 // btn_shaper(clk, rst, b_in, b_out)
150 // Authentication(Digit, B_in_shaped, Logout_Req_from_GameCtrl, clk, reset,
151 // LoggedIn, LoggedOut, ID_passed,
152 // PlayerID_to_scoreRAM, NumberID_to_scoreRAM)
153 // gameControl(clk, rst, timer_done, id_log_inout, game_start_key,
154 // game_control_key, verified, id_passed, difficulty_in,
155 // timer_enable, timer_reconfig, enable_mega_compar,
156 // pulse_w_ram, pulse_sctrl_ram, difficulty_out,
157 // logout_req, score_soft_rst, display_mode)
158 // twoDigitTimer(clk, rst, time_reconfig, timer_enable,
159 // ones_count, tens_count, timer_done)
160 // megaComparator(clk, rst, enable, difficulty, score_soft_rst,
161 // player_in, player_load_in, score, display_random)
162 // scoreRAM_top(pulse_WRAM_from_GameCtrl, pulse_sctrl_RAM_from_GameCtrl,
163 // PlayerID_from_Auth, NumberID_from_Auth, LatestScore,
164 // clk, reset, MyBest_to_DispCnt, GlobalBest_to_DispCnt)
165 // ScoreRAM_adapter(clk, rst, pulse_w_ram, pulse_sctrl_ram,
166 // my_best, global_best, lboard_entry)
167 // DisplayCnt(clk, rst, display_mode, ones_count, tens_count,
168 // DisplayRandom, lboard_entry, HEX0..HEX5)
169 // =====
170

```

```

170
171
172 // =====
173 // Button shapers -- KEY[0] is reset (no shape), KEY[1..3] shaped
174 // =====
175 btn_shaper u_bs_key1 (
176     .clk    (CLOCK_50),
177     .rst    (KEY[0]),
178     .b_in   (KEY[1]),
179     .b_out  (key1_shaped)
180 );
181
182 btn_shaper u_bs_key2 (
183     .clk    (CLOCK_50),
184     .rst    (KEY[0]),
185     .b_in   (KEY[2]),
186     .b_out  (key2_shaped)
187 );
188
189 btn_shaper u_bs_key3 (
190     .clk    (CLOCK_50),
191     .rst    (KEY[0]),
192     .b_in   (KEY[3]),
193     .b_out  (key3_shaped)
194 );
195
196
197 // =====
198 // Authentication -- 4-digit ID + 4-digit PW match
199 //   Digit feed comes from SW[9:6] (digit_auth), separate from
200 //   the gameplay SW[3:0] feed. B_in_shaped is the KEY[3]
201 //   shaped pulse.
202 // =====
203 Authentication u_auth (
204     .Digit                (digit_auth),
205     .B_in_shaped          (key2_shaped),

```



```

203 Authentication u_auth (
204     .Digit                (digit_auth),
205     .B_in_shaped          (key2_shaped),
206     .Logout_Req_from_GameCtrl (logout_req),
207     .clk                  (CLOCK_50),
208     .reset                (KEY[0]),
209     .LoggedIn             (LoggedIn),
210     .LoggedOut            (LoggedOut),
211     .ID_passed            (ID_passed),
212     .PlayerID_to_scoreRAM (PlayerID_to_scoreRAM),
213     .NumberID_to_scoreRAM (NumberID_to_scoreRAM)
214 );
215
216
217 // =====
218 // gameControl -- master FSM. Drives display_mode (one-hot phase)
219 // to both DisplayCnt and LEDR[6:0]. game_control_key shares
220 // the KEY[1] shaped pulse with player_load_in.
221 // =====
222 gameControl u_ctrl (
223     .clk                (CLOCK_50),
224     .rst                (KEY[0]),
225     .timer_done         (timer_done),
226     .id_log_inout       (key3_shaped),           // raw KEY[3], active-LOW
227     .game_start_key     (key2_shaped),
228     .game_control_key   (key1_shaped),
229     .verified           (LoggedIn),
230     .id_passed          (ID_passed),
231     .difficulty_in      (difficulty),
232     .timer_enable       (timer_enable),
233     .timer_reconfig     (timer_reconfig),
234     .enable_mega_compar (enable_mega_compar),
235     .pulse_w_ram        (pulse_w_ram),
236     .pulse_scri_ram     (pulse_scri_ram),
237     .difficulty_out     (difficulty_out),
238     .logout_req         (logout_req),
239     .score_soft_rst     (score_soft_rst),
240     .display_mode       (display_mode)
241 );

```

```

238     .logout_req      (logout_req),
239     .score_soft_rst   (score_soft_rst),
240     .display_mode     (display_mode)
241 );
242
243
244 // =====
245 // twoDigitTimer -- 99 -> 00 countdown
246 //   Named-port bridge: gameControl drives "timer_reconfig"
247 //   but twoDigitTimer's input is "time_reconfig" (no 'r').
248 // =====
249 twoDigitTimer u_timer (
250     .clk      (CLOCK_50),
251     .rst      (KEY[0]),
252     .time_reconfig (timer_reconfig),    // <-- name bridge
253     .timer_enable (timer_enable),
254     .ones_count   (ones_count),
255     .tens_count   (tens_count),
256     .timer_done   (timer_done)
257 );
258
259
260 // =====
261 // megaComparator -- gameplay engine (RNG + slicing + match +
262 // saturating score). player_load_in shares KEY[1] with the
263 // leaderboard scroll signal.
264 // =====
265 megaComparator u_mega (
266     .clk      (CLOCK_50),
267     .rst      (KEY[0]),
268     .enable    (enable_mega_compar),
269     .difficulty (difficulty_out),
270     .score_soft_rst (score_soft_rst),
271     .player_in   (digit_play),    // SW[3:0]
272     .player_load_in (key1_shaped),
273     .score       (score),
274     .display_random (display_random)
275 );
276
277
278 //

```

```

277
278 // =====
279 // scoreRAM_top -- per-player and global high-score storage
280 //   (Tru's module; ZM bug fixes applied: port width,
281 //     blocking->non-blocking, MyBest_reg refresh,
282 //     BestPlayer_reg init, RAMINIT data 0x000000.)
283 // =====
284 scoreRAM_top u_scoreram (
285     .pulse_WRAM_from_GameCtrl    (pulse_w_ram),
286     .pulse_sctrl_RAM_from_GameCtrl (pulse_sctrl_ram),
287     .PlayerID_from_Auth           (PlayerID_to_scoreRAM),
288     .NumberID_from_Auth           (NumberID_to_scoreRAM),
289     .LatestScore                  (score),
290     .clk                          (CLOCK_50),
291     .reset                        (KEY[0]),
292     .MyBest_to_DispCnt            (my_best),
293     .GlobalBest_to_DispCnt        (global_best)
294 );
295
296
297 // =====
298 // ScoreRAM_adapter -- bridges Tru's 2 parallel outputs
299 //   (MyBest + GlobalBest) to DisplayCnt's single lboard_entry
300 //   input. Internal 1-bit toggle flips on scroll pulse.
301 // =====
302 ScoreRAM_adapter u_adapter (
303     .clk          (CLOCK_50),
304     .rst          (KEY[0]),
305     .pulse_w_ram   (pulse_w_ram),
306     .pulse_sctrl_ram (pulse_sctrl_ram),
307     .my_best       (my_best),
308     .global_best   (global_best),
309     .lboard_entry  (lboard_entry)
310 );
311
312
313 // =====
314 // DisplayCnt -- HEX renderer; layout switches on display_mode
315 // =====
316 DisplayCnt u_disp (

```

```

313 // =====
314 // DisplayCnt -- HEX renderer; layout switches on display_mode
315 // =====
316 DisplayCnt u_disp (
317     .clk          (CLOCK_50),
318     .rst          (KEY[0]),
319     .display_mode (display_mode),
320     .ones_count   (ones_count),
321     .tens_count   (tens_count),
322     .DisplayRandom (display_random),
323     .lboard_entry (lboard_entry),
324     .HEX0         (HEX0),
325     .HEX1         (HEX1),
326     .HEX2         (HEX2),
327     .HEX3         (HEX3),
328     .HEX4         (HEX4),
329     .HEX5         (HEX5)
330 );
331
332
333 // =====
334 // LED phase indicator
335 // =====
336 assign LEDR[6:0] = display_mode;
337 reg [7:0] score_at_write;
338 always @(posedge CLOCK_50) begin
339     if (KEY[0] == 1'b0)
340         score_at_write <= 8'd0;
341     else if (pulse_w_ram == 1'b1)
342         score_at_write <= score; // snapshot score the cycle write fires
343 end
344
345 assign LEDR[7] = score_at_write[0];
346 assign LEDR[8] = score_at_write[1];
347 assign LEDR[9] = score_at_write[2];
348
349 endmodule
350

```

Authentication Module

```

1
2 module Authentication(Digit, B_in_shaped, Logout_Req_from_GameCtrl, clk, reset,
3     LoggedIn, LoggedOut, ID_passed, PlayerID_to_scoreRAM, NumberID_to_scoreRAM);
4
5     input [3:0] Digit;
6     input B_in_shaped, Logout_Req_from_GameCtrl, clk, reset;
7
8     output LoggedIn, LoggedOut, ID_passed;
9     output [4:0] PlayerID_to_scoreRAM;
10    output [15:0] NumberID_to_scoreRAM;
11
12    wire matchedID, Logout_Req_between_ID_FW;
13    wire [4:0] PlayerID_between_ID_FW;
14    wire [15:0] NumberID_between_ID_FW;
15
16    PW_Auth PW_Auth_top(matchedID, Digit, B_in_shaped, PlayerID_between_ID_FW, NumberID_between_ID_FW,
17        Logout_Req_from_GameCtrl, clk, reset, LoggedIn, LoggedOut, PlayerID_to_scoreRAM, NumberID_to_scoreRAM, Logout_Req_between_ID_FW);
18
19    ID_Auth ID_Auth_top(Digit, B_in_shaped, Logout_Req_between_ID_FW, clk, reset, matchedID, ID_passed,
20        PlayerID_between_ID_FW, NumberID_between_ID_FW);
21
22 endmodule

```

Btn Shaper Module

```

34  L*/
35  module btn_shaper(clk, rst, b_in, b_out);
36      input  clk, rst, b_in;
37      output reg b_out;           // active-HIGH 1-cycle pulse on press
38
39      parameter HIGH = 1'b1;
40      parameter LOW  = 1'b0;
41
42      parameter IDLE  = 2'd0;
43      parameter PULSE = 2'd1;
44      parameter WAIT  = 2'd2;
45
46      reg [1:0] state;
47
48      always @(posedge clk) begin
49          if (rst == LOW) begin
50              state <= IDLE;
51              b_out <= LOW;
52          end
53          else begin
54              case (state)
55                  IDLE: begin
56                      b_out <= LOW;
57                      // raw b_in is active-LOW; press = HIGH->LOW edge
58                      if (b_in == LOW) begin
59                          state <= PULSE;
60                      end
61                  end
62
63                  PULSE: begin
64                      b_out <= HIGH;           // single-cycle output
65                      state <= WAIT;
66                  end
67
68                  WAIT: begin
69                      b_out <= LOW;
70                      if (b_in == HIGH) begin // re-arm after release
71                          state <= IDLE;
72                      end
73                  end
74
75                  default: begin
76                      state <= IDLE;
77                      b_out <= LOW;
78                  end
79              endcase
80          end
81      end
82  endmodule

```

Comparator Module

```

1  module Comparator(rst, clk, Enable, GameDifficulty, Player_in, PlayerLoad_in, ScorePulse, DisplayRandom);
2      input rst, clk, Enable, PlayerLoad_in;
3      input [1:0] GameDifficulty;
4      input [3:0] Player_in;
5
6      output ScorePulse;
7      output [3:0] DisplayRandom;
8
9      reg ScorePulse;
10     reg [3:0] DisplayRandom;
11
12     reg [15:0] FullPlayer_in;
13     reg Random_req;
14     reg soft_rst;
15
16     reg [1:0] DisplayCount;
17     reg [1:0] PlayerCount;
18
19     wire [15:0] RandomNum;
20     wire DisplayTime;
21
22     reg [15:0] RNG;
23     reg DisplayTimeOut;
24
25     reg [3:0] State;
26
27     parameter Init=0, RequestedRNG=1, CatchRNG=2, DisplayRNG=3, CatchPlayerInput=4, Compare=5, PulseToScoreTracker=6;
28
29     GameTimer RNGDisplayTimer(rst, clk, Enable, soft_rst, GameDifficulty, DisplayTime);
30     RNG_Comparator RandomGenerator(rst, clk, Random_req, RandomNum);
31
32     always @(posedge clk)
33     begin
34         if (rst == 1'b0) begin
35             ScorePulse <= 1'b0;
36             DisplayRandom <= 4'b1000;
37             FullPlayer_in <= 16'b0000_0000_0000_0000;
38             Random_req <= 1'b0;
39             soft_rst <= 1'b0;
40             DisplayCount <= 3'b000;
41             PlayerCount <= 3'b000;
42             State <= Init;
43         end
44
45         // =====
46         // Enable-drop guard (Rev 2 fix)
47         // Without this, the FSM stays in whatever state it was in
48         // (often CatchPlayerInput) when gameControl drops
49         // enable_mega_compar at END_GAME or on logout. The next
50         // round then picks up with stale player input and stale
51         // RNG, producing a corrupted round. Force-return to Init
52         // so each new round starts from a known state.
53         // =====
54         else if (Enable == 1'b0) begin
55             ScorePulse <= 1'b0;
56             DisplayRandom <= 4'b1000;
57             FullPlayer_in <= 16'b0000_0000_0000_0000;
58             Random_req <= 1'b0;
59             soft_rst <= 1'b0;

```

Comparator.v

```

58 Random_req <= 1'b0;
59 soft_rst <= 1'b0;
60 DisplayCount <= 2'b00;
61 PlayerCount <= 2'b00;
62 State <= Init;
63 end
64 else begin
65     case (State)
66     Init: begin
67         ScorePulse <= 1'b0;
68         DisplayRandom <= 4'b1000;
69         FullPlayer_in <= 16'b0000_0000_0000_0000;
70         Random_req <= 1'b0;
71         soft_rst <= 1'b0;
72         DisplayCount <= 2'b00;
73         PlayerCount <= 2'b00;
74         if (Enable == 1'b1) begin
75             Random_req <= 1'b1;
76             State <= RequestedRNG;
77         end
78         else begin
79             State <= Init;
80         end
81     end
82
83     RequestedRNG: begin
84         Random_req <= 1'b0;
85         State <= CatchRNG;
86     end
87
88     CatchRNG: begin
89         RNG <= RandomNum;
90         soft_rst <= 1'b1;
91         State <= DisplayRNG;
92     end
93
94     DisplayRNG: begin
95         soft_rst <= 1'b0;
96         DisplayTimeOut <= DisplayTime;
97         case (DisplayCount)
98         2'b00: begin
99             DisplayRandom <= {RNG[1], RNG[9], RNG[13], RNG[12]};
100         end
101         2'b01: begin
102             DisplayRandom <= {RNG[11], RNG[2], RNG[6], RNG[10]};
103         end
104         2'b10: begin
105             DisplayRandom <= {RNG[7], RNG[15], RNG[3], RNG[4]};
106         end
107         2'b11: begin
108             DisplayRandom <= {RNG[5], RNG[14], RNG[8], RNG[0]};
109         end
110         default: begin
111             DisplayRandom <= 4'b1000;
112         end
113     endcase
114     if (DisplayTimeOut == 1'b1 && DisplayCount == 2'b11)
115     begin
116         DisplayRandom <= 4'b1000;

```

```

Comparator.v
115     begin
116         DisplayRandom <= 4'b1000;
117         State <= CatchPlayerInput;
118     end
119 else begin
120     if (DisplayTimeOut == 1'b1) begin
121         DisplayCount <= DisplayCount + 1;
122         State <= DisplayRNG;
123     end
124     else
125         State <= DisplayRNG;
126     end
127 end
128
129 CatchPlayerInput: begin
130     if (PlayerLoad_in == 1'b1) begin
131         case (PlayerCount)
132             2'b00: begin
133                 FullPlayer_in[15:12] <= Player_in;
134             end
135             2'b01: begin
136                 FullPlayer_in[11:8] <= Player_in;
137             end
138             2'b10: begin
139                 FullPlayer_in[7:4] <= Player_in;
140             end
141             2'b11: begin
142                 FullPlayer_in[3:0] <= Player_in;
143             end
144             default: begin
145                 FullPlayer_in <= 16'h0000;
146             end
147         endcase
148
149         if (PlayerCount == 2'b11)
150             State <= Compare;
151         else
152             PlayerCount <= PlayerCount + 1;
153         end
154     end
155 end
156 Compare: begin
157     if ( RNG == FullPlayer_in)
158         ScorePulse <= 1'b1;
159
160     State <= PulseToScoreTracker;
161 end
162
163 PulseToScoreTracker: begin
164     ScorePulse <= 1'b0;
165     State <= Init;
166 end
167
168 default: begin
169     ScorePulse <= 1'b0;
170     DisplayRandom <= 4'b1000;
171     FullPlayer_in <= 16'b0000_0000_0000_0000;
172     Random_req <= 1'b0;
173     soft rst <= 1'b0;
174     soft_rst <= 1'b0;
175     DisplayCount <= 2'b00;
176     PlayerCount <= 2'b00;
177
178     State <= Init;
179 end
180 endcase
181 end
182 end
183
184 endmodule

```


countTo2 Module

```
countTo2.v
1  module countTo2(clk, rst, enable, timeout);
2      input wire clk, rst, enable;
3      output reg timeout;
4      reg [3:0] count = 4'h0;
5
6      always @ (posedge clk or negedge rst) begin
7          if (rst == 0) begin                // resting the count and time out
8              count <= 4'h0;
9              timeout <= 0;
10         end
11         else begin
12             if (enable == 1) begin // only counting if the enable is one
13                 if (count >= 4'h9) begin // looking for count to be over 1
14                     count <= 4'h0;
15                     timeout <= 1;
16                 end else begin // increase the count with clock edge if
17                     count <= count + 1;
18                     timeout <= 0;
19                 end
20             end else begin // looking for edge case scenario
21                 count <= count;
22                 timeout <= 0;
23             end
24         end
25     end
26 endmodule
```

countTo3

```
countTo3.v
1  module countTo3(clk, rst, enable, timeout);
2      input wire clk, rst, enable;
3      output reg timeout;
4      reg [6:0] count = 7'h00;
5
6      always @ (posedge clk or negedge rst) begin
7          if (rst == 0) begin                // resting the count and time out
8              count <= 7'h00;
9              timeout <= 0;
10         end
11         else begin
12             if (enable == 1) begin // only counting if the enable is one
13                 if (count >= 7'h63) begin // looking for count to be over 2
14                     count <= 7'h00;
15                     timeout <= 1;
16                 end else begin // increase the count with clock edge if
17                     count <= count + 1;
18                     timeout <= 0;
19                 end
20             end else begin // looking for edge case scenario
21                 count <= count;
22                 timeout <= 0;
23             end
24         end
25     end
26 endmodule
```

countTo100

```

22 module CountTo100 (OneMSSignal, rst, soft_rst, clk, HundredOut);
23     input  OneMSSignal, rst, clk, soft_rst;
24     output HundredOut;
25     reg    HundredOut;
26     reg [7:0] Counter;
27
28     always @(posedge clk)
29     begin
30         // Active-LOW hard reset
31         if (rst == 1'b0)
32         begin
33             HundredOut <= 1'b0;
34             Counter    <= 8'b0000_0000;
35         end
36     else
37     begin
38         // Default: HundredOut idles LOW.
39         // Use non-blocking <= here to avoid mixing with
40         // the <= overrides below -- mixing blocking and
41         // non-blocking on the same reg in one always block
42         // is a synthesis hazard. (Was '=' previously.)
43         HundredOut <= 1'b0;
44
45         // Per-round soft reset from Comparator.CatchRNG.
46         // Clears the count register too, so the next round
47         // starts from 0 (not from a partial count carried
48         // over from the previous round).
49         if (soft_rst == 1'b1) begin
50             HundredOut <= 1'b0;
51             Counter    <= 8'b0000_0000;
52         end
53
54         // Increment on each 1 ms upstream tick;
55         // pulse HundredOut on the 100th tick and wrap.
56         if (OneMSSignal == 1'b1)
57         begin
58             Counter <= Counter + 1;
59             //if (Counter == 8'b0000_0011) // sim-speed value
60             if (Counter == 8'b0110_0100) // 100 in production
61             begin
62                 HundredOut <= 1'b1;
63                 Counter    <= 8'b0000_0000;
64             end
65         end
66     else
67     begin
68         //Nothing
69     end
70 end
71 end
72 endmodule

```

digitTimer Module

```
digitTimer.v
1  /*
2  Course: ECE 6370 Advanced Digital Design
3  Author: Zaki Mir
4  PeopleSoft ID: 1675819
5  Porject: digitTimer
6  Project Discription:
7  This module implements an FPGA-based count down system.
8  the module will be counting down from 9 to 0. insted
9  of resetting to 0 it would wrap back to 0.
10 */
11
12 module digitTimer(rst, enable, reconfig, borrowDwn, noBorrowUp, borrowUp, noBorrowDwn, count);
13 input rst, enable, reconfig, borrowDwn, noBorrowUp;
14 output reg borrowUp, noBorrowDwn;
15 output reg [3:0] count;
16
17 parameter nine = 4'b1001, eight = 4'b1000, seven = 4'b0111, six = 4'b0110, five = 4'b0101,
18         four = 4'b0100, three = 4'b0011, two = 4'b0010, one = 4'b0001, zero = 4'b0000;
19
20 always @(posedge borrowDwn or negedge rst or posedge reconfig) begin
21     if(rst == 1'b0) begin
22         count <= zero;
23         borrowUp <= 1'b0;
24         noBorrowDwn <= 1'b0;
25     end
26     else begin
27         if (reconfig == 1'b1) begin
28             count <= nine;
29             borrowUp <= 1'b0;
30             noBorrowDwn <= 1'b0;
31         end
32         else if (enable == 1'b1) begin
33             case(count)
34                 nine: begin
35                     count <= eight;
36                     borrowUp <= 1'b0;
37                 end
38                 eight: begin
39                     count <= seven;
40                 end
41                 seven: begin
42                     count <= six;
43                 end
44                 six: begin
45                     count <= five;
46                 end
47                 five: begin
48                     count <= four;
49                 end
50                 four: begin
51                     count <= three;
52                 end
53                 three: begin
54                     count <= two;
```

```
digitTimer.v
52      end
53      three: begin
54          count <= two;
55      end
56      two: begin
57          count <= one;
58      end
59      one: begin
60          count <= zero;
61          noBorrowDwn <= 1'b1;
62      end
63      zero: begin
64          // request to borrow
65
66          if(noBorrowUp == 0) begin
67              count <= nine;
68              borrowUp <= 1'b1;
69
70          end else begin
71              borrowUp <= 1'b0;
72
73          end
74      end
75      default: begin
76          count <= count;
77          borrowUp <= 1'b0;
78
79      end
80  endcase
81  end
82  else begin
83      borrowUp <= 1'b0;
84      noBorrowDwn <= 1'b0;
85      count <= zero;
86  end
87  end
88  end
89
90  endmodule
```

DisplayCnt Module

```

58 module DisplayCnt(
59     clk, rst,
60     display_mode,
61     ones_count, tens_count,
62     DisplayRandom,
63     lboard_entry,
64     HEX0, HEX1, HEX2, HEX3, HEX4, HEX5
65 );
66 // ----- System -----
67 input clk, rst;
68
69 // ----- One-hot phase from gameControl -----
70 input [6:0] display_mode;
71
72 // ----- Game data -----
73 input [3:0] ones_count, tens_count; // from twoDigitTimer
74 input [3:0] DisplayRandom; // from Comparator
75 input [23:0] lboard_entry; // from ScoreRAM, packed {ID[15:0], score[7:0]}
76
77 // ----- DE0-CV 7-seg outputs (active-LOW) -----
78 output reg [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5;
79
80 // ----- One-hot mode patterns (match gameControl) -----
81 parameter MODE_UNAUTHED = 7'b000_0001;
82 parameter MODE_ID_IN = 7'b000_0010;
83 parameter MODE_AUTHED = 7'b000_0100;
84 parameter MODE_PRE_GAME = 7'b000_1000;
85 parameter MODE_PLAYING = 7'b001_0000;
86 parameter MODE_END_GAME = 7'b010_0000;
87 parameter MODE_LBOARD = 7'b100_0000;
88
89 // ----- Logic-level aliases -----
90 parameter HIGH = 1'b1;
91 parameter LOW = 1'b0;
92
93 // ----- 7-seg patterns (active-LOW; bit[6]=g, bit[0]=a) -----
94 parameter SEG_0 = 7'b1000000;
95 parameter SEG_1 = 7'b1111001;
96 parameter SEG_2 = 7'b0100100;
97 parameter SEG_3 = 7'b0110000;
98 parameter SEG_4 = 7'b0011001;
99 parameter SEG_5 = 7'b0010010;
100 parameter SEG_6 = 7'b0000010;
101 parameter SEG_7 = 7'b1111000;
102 parameter SEG_8 = 7'b0000000;
103 parameter SEG_9 = 7'b0010000;
104 parameter SEG_A = 7'b0001000;
105 parameter SEG_B = 7'b0000011;
106 parameter SEG_C = 7'b1000110;
107 parameter SEG_D = 7'b0100001;
108 parameter SEG_E = 7'b0000110;
109 parameter SEG_F = 7'b0001110;
110 parameter SEG_BLANK = 7'b1111111;

```

```
DisplayCntv ✕
109 parameter SEG_F      = 7'b0001110;
110 parameter SEG_BLANK = 7'b1111111;
111
112 // =====
113 // Single-always-block FSM
114 //   Cases on display_mode (one-hot from gameControl).
115 //   Each branch drives all six HEX outputs.
116 //   Hex-to-7seg conversion is inlined as nested case
117 //   statements (no helper function).
118 // =====
119 always @(posedge clk) begin
120     if (rst == LOW) begin
121         HEX0 <= SEG_BLANK;
122         HEX1 <= SEG_BLANK;
123         HEX2 <= SEG_BLANK;
124         HEX3 <= SEG_BLANK;
125         HEX4 <= SEG_BLANK;
126         HEX5 <= SEG_BLANK;
127     end
128     else begin
129         case (display_mode)
130
131             // =====
132             // UNAUTHED / ID_IN / AUTHED
133             //   Pre-difficulty phases -- all HEX blank.
134             //   Player phase is communicated via the LEDs
135             //   (driven from display_mode at top level).
136             // =====
137             MODE_UNAUTHED, MODE_ID_IN, MODE_AUTHED: begin
138                 HEX0 <= SEG_BLANK;
139                 HEX1 <= SEG_BLANK;
140                 HEX2 <= SEG_BLANK;
141                 HEX3 <= SEG_BLANK;
142                 HEX4 <= SEG_BLANK;
143                 HEX5 <= SEG_BLANK;
144             end
145
146             // =====
147             // PRE_GAME
148             //   Show "99" on HEX5/HEX4 as a preview of the
149             //   round timer. Other HEX blank.
150             // =====
151             MODE_PRE_GAME: begin
152                 HEX5 <= SEG_9;
153                 HEX4 <= SEG_9;
154                 HEX3 <= SEG_BLANK;
155                 HEX2 <= SEG_BLANK;
156                 HEX1 <= SEG_BLANK;
157                 HEX0 <= SEG_BLANK;
158             end
159
160             // =====
161             // PLAYING
162             //   HEX5/HEX4 = round timer (tens, ones).
```

DisplayCnt.v

```
160 // =====
161 // PLAYING
162 //   HEX5/HEX4 = round timer (tens, ones).
163 //   HEX0 = current Comparator RNG nibble.
164 //   HEX3/HEX2/HEX1 stay blank.
165 // =====
166 MODE_PLAYING: begin
167   HEX3 <= SEG_BLANK;
168   HEX2 <= SEG_BLANK;
169   HEX1 <= SEG_BLANK;
170
171   // HEX5: tens_count hex decode
172   case (tens_count)
173     4'h0:   HEX5 <= SEG_0;
174     4'h1:   HEX5 <= SEG_1;
175     4'h2:   HEX5 <= SEG_2;
176     4'h3:   HEX5 <= SEG_3;
177     4'h4:   HEX5 <= SEG_4;
178     4'h5:   HEX5 <= SEG_5;
179     4'h6:   HEX5 <= SEG_6;
180     4'h7:   HEX5 <= SEG_7;
181     4'h8:   HEX5 <= SEG_8;
182     4'h9:   HEX5 <= SEG_9;
183     4'hA:   HEX5 <= SEG_A;
184     4'hB:   HEX5 <= SEG_B;
185     4'hC:   HEX5 <= SEG_C;
186     4'hD:   HEX5 <= SEG_D;
187     4'hE:   HEX5 <= SEG_E;
188     4'hF:   HEX5 <= SEG_F;
189     default: HEX5 <= SEG_BLANK;
190   endcase
191
192   // HEX4: ones_count hex decode
193   case (ones_count)
194     4'h0:   HEX4 <= SEG_0;
195     4'h1:   HEX4 <= SEG_1;
196     4'h2:   HEX4 <= SEG_2;
197     4'h3:   HEX4 <= SEG_3;
198     4'h4:   HEX4 <= SEG_4;
199     4'h5:   HEX4 <= SEG_5;
200     4'h6:   HEX4 <= SEG_6;
201     4'h7:   HEX4 <= SEG_7;
202     4'h8:   HEX4 <= SEG_8;
203     4'h9:   HEX4 <= SEG_9;
204     4'hA:   HEX4 <= SEG_A;
205     4'hB:   HEX4 <= SEG_B;
206     4'hC:   HEX4 <= SEG_C;
207     4'hD:   HEX4 <= SEG_D;
208     4'hE:   HEX4 <= SEG_E;
209     4'hF:   HEX4 <= SEG_F;
210     default: HEX4 <= SEG_BLANK;
211   endcase
212
213   // HEX0: DisplayRandom hex decode
```

```
211 endcase
212
213 // HEX0: DisplayRandom hex decode
214 case (DisplayRandom)
215     4'h0:  HEX0 <= SEG_0;
216     4'h1:  HEX0 <= SEG_1;
217     4'h2:  HEX0 <= SEG_2;
218     4'h3:  HEX0 <= SEG_3;
219     4'h4:  HEX0 <= SEG_4;
220     4'h5:  HEX0 <= SEG_5;
221     4'h6:  HEX0 <= SEG_6;
222     4'h7:  HEX0 <= SEG_7;
223     4'h8:  HEX0 <= SEG_8;
224     4'h9:  HEX0 <= SEG_9;
225     4'hA:  HEX0 <= SEG_A;
226     4'hB:  HEX0 <= SEG_B;
227     4'hC:  HEX0 <= SEG_C;
228     4'hD:  HEX0 <= SEG_D;
229     4'hE:  HEX0 <= SEG_E;
230     4'hF:  HEX0 <= SEG_F;
231     default: HEX0 <= SEG_BLANK;
232 endcase
233 end
234
235 // =====
236 // END_GAME
237 // 1-second pause before LBOARD. Shows the
238 // timer at its final value (which is 00 since
239 // timer_done just fired). Other HEX blank.
240 // =====
241 MODE_END_GAME: begin
242     HEX3 <= SEG_BLANK;
243     HEX2 <= SEG_BLANK;
244     HEX1 <= SEG_BLANK;
245     HEX0 <= SEG_BLANK;
246
247 // HEX5: tens_count hex decode (will be 0)
248 case (tens_count)
249     4'h0:  HEX5 <= SEG_0;
250     4'h1:  HEX5 <= SEG_1;
251     4'h2:  HEX5 <= SEG_2;
252     4'h3:  HEX5 <= SEG_3;
253     4'h4:  HEX5 <= SEG_4;
254     4'h5:  HEX5 <= SEG_5;
255     4'h6:  HEX5 <= SEG_6;
256     4'h7:  HEX5 <= SEG_7;
257     4'h8:  HEX5 <= SEG_8;
258     4'h9:  HEX5 <= SEG_9;
259     4'hA:  HEX5 <= SEG_A;
260     4'hB:  HEX5 <= SEG_B;
261     4'hC:  HEX5 <= SEG_C;
262     4'hD:  HEX5 <= SEG_D;
263     4'hE:  HEX5 <= SEG_E;
264     4'hF:  HEX5 <= SEG_F;
```



```
DisplayCnt.v
262      4'hD:    HEX5 <= SEG_D;
263      4'hE:    HEX5 <= SEG_E;
264      4'hF:    HEX5 <= SEG_F;
265      default: HEX5 <= SEG_BLANK;
266  endcase
267
268  // HEX4: ones_count hex decode (will be 0)
269  case (ones_count)
270      4'h0:    HEX4 <= SEG_0;
271      4'h1:    HEX4 <= SEG_1;
272      4'h2:    HEX4 <= SEG_2;
273      4'h3:    HEX4 <= SEG_3;
274      4'h4:    HEX4 <= SEG_4;
275      4'h5:    HEX4 <= SEG_5;
276      4'h6:    HEX4 <= SEG_6;
277      4'h7:    HEX4 <= SEG_7;
278      4'h8:    HEX4 <= SEG_8;
279      4'h9:    HEX4 <= SEG_9;
280      4'hA:    HEX4 <= SEG_A;
281      4'hB:    HEX4 <= SEG_B;
282      4'hC:    HEX4 <= SEG_C;
283      4'hD:    HEX4 <= SEG_D;
284      4'hE:    HEX4 <= SEG_E;
285      4'hF:    HEX4 <= SEG_F;
286      default: HEX4 <= SEG_BLANK;
287  endcase
288  end
289
290  // =====
291  // LBOARD
292  // 24-bit lboard_entry sliced into 6 nibbles,
293  // one per HEX. Format:
294  //   HEX5/4/3/2 = 4-digit ID (entry[23:8])
295  //   HEX1/0     = 2-digit score (entry[7:0])
296  // =====
297  MODE_LBOARD: begin
298      // HEX5: lboard_entry[23:20] -- ID digit 1
299      case (lboard_entry[23:20])
300          4'h0:    HEX5 <= SEG_0;
301          4'h1:    HEX5 <= SEG_1;
302          4'h2:    HEX5 <= SEG_2;
303          4'h3:    HEX5 <= SEG_3;
304          4'h4:    HEX5 <= SEG_4;
305          4'h5:    HEX5 <= SEG_5;
306          4'h6:    HEX5 <= SEG_6;
307          4'h7:    HEX5 <= SEG_7;
308          4'h8:    HEX5 <= SEG_8;
309          4'h9:    HEX5 <= SEG_9;
310          4'hA:    HEX5 <= SEG_A;
311          4'hB:    HEX5 <= SEG_B;
312          4'hC:    HEX5 <= SEG_C;
313          4'hD:    HEX5 <= SEG_D;
314          4'hE:    HEX5 <= SEG_E;
315          4'hF:    HEX5 <= SEG_F;
```

```

313         4'hD:    HEX5 <= SEG_D;
314         4'hE:    HEX5 <= SEG_E;
315         4'hF:    HEX5 <= SEG_F;
316         default: HEX5 <= SEG_BLANK;
317     endcase
318
319     // HEX4: lboard_entry[19:16] -- ID digit 2
320     case (lboard_entry[19:16])
321         4'h0:    HEX4 <= SEG_0;
322         4'h1:    HEX4 <= SEG_1;
323         4'h2:    HEX4 <= SEG_2;
324         4'h3:    HEX4 <= SEG_3;
325         4'h4:    HEX4 <= SEG_4;
326         4'h5:    HEX4 <= SEG_5;
327         4'h6:    HEX4 <= SEG_6;
328         4'h7:    HEX4 <= SEG_7;
329         4'h8:    HEX4 <= SEG_8;
330         4'h9:    HEX4 <= SEG_9;
331         4'hA:    HEX4 <= SEG_A;
332         4'hB:    HEX4 <= SEG_B;
333         4'hC:    HEX4 <= SEG_C;
334         4'hD:    HEX4 <= SEG_D;
335         4'hE:    HEX4 <= SEG_E;
336         4'hF:    HEX4 <= SEG_F;
337         default: HEX4 <= SEG_BLANK;
338     endcase
339
340     // HEX3: lboard_entry[15:12] -- ID digit 3
341     case (lboard_entry[15:12])
342         4'h0:    HEX3 <= SEG_0;
343         4'h1:    HEX3 <= SEG_1;
344         4'h2:    HEX3 <= SEG_2;
345         4'h3:    HEX3 <= SEG_3;
346         4'h4:    HEX3 <= SEG_4;
347         4'h5:    HEX3 <= SEG_5;
348         4'h6:    HEX3 <= SEG_6;
349         4'h7:    HEX3 <= SEG_7;
350         4'h8:    HEX3 <= SEG_8;
351         4'h9:    HEX3 <= SEG_9;
352         4'hA:    HEX3 <= SEG_A;
353         4'hB:    HEX3 <= SEG_B;
354         4'hC:    HEX3 <= SEG_C;
355         4'hD:    HEX3 <= SEG_D;
356         4'hE:    HEX3 <= SEG_E;
357         4'hF:    HEX3 <= SEG_F;
358         default: HEX3 <= SEG_BLANK;
359     endcase
360
361     // HEX2: lboard_entry[11:8] -- ID digit 4
362     case (lboard_entry[11:8])
363         4'h0:    HEX2 <= SEG_0;
364         4'h1:    HEX2 <= SEG_1;
365         4'h2:    HEX2 <= SEG_2;
366         4'h3:    HEX2 <= SEG_3;

```

```

364      4'h1:    HEX2 <= SEG_1;
365      4'h2:    HEX2 <= SEG_2;
366      4'h3:    HEX2 <= SEG_3;
367      4'h4:    HEX2 <= SEG_4;
368      4'h5:    HEX2 <= SEG_5;
369      4'h6:    HEX2 <= SEG_6;
370      4'h7:    HEX2 <= SEG_7;
371      4'h8:    HEX2 <= SEG_8;
372      4'h9:    HEX2 <= SEG_9;
373      4'hA:    HEX2 <= SEG_A;
374      4'hB:    HEX2 <= SEG_B;
375      4'hC:    HEX2 <= SEG_C;
376      4'hD:    HEX2 <= SEG_D;
377      4'hE:    HEX2 <= SEG_E;
378      4'hF:    HEX2 <= SEG_F;
379      default: HEX2 <= SEG_BLANK;
380  endcase
381
382  // HEX1: lboard_entry[7:4] -- score digit 1 (tens)
383  case (lboard_entry[7:4])
384      4'h0:    HEX1 <= SEG_0;
385      4'h1:    HEX1 <= SEG_1;
386      4'h2:    HEX1 <= SEG_2;
387      4'h3:    HEX1 <= SEG_3;
388      4'h4:    HEX1 <= SEG_4;
389      4'h5:    HEX1 <= SEG_5;
390      4'h6:    HEX1 <= SEG_6;
391      4'h7:    HEX1 <= SEG_7;
392      4'h8:    HEX1 <= SEG_8;
393      4'h9:    HEX1 <= SEG_9;
394      4'hA:    HEX1 <= SEG_A;
395      4'hB:    HEX1 <= SEG_B;
396      4'hC:    HEX1 <= SEG_C;
397      4'hD:    HEX1 <= SEG_D;
398      4'hE:    HEX1 <= SEG_E;
399      4'hF:    HEX1 <= SEG_F;
400      default: HEX1 <= SEG_BLANK;
401  endcase
402
403  // HEX0: lboard_entry[3:0] -- score digit 2 (ones)
404  case (lboard_entry[3:0])
405      4'h0:    HEX0 <= SEG_0;
406      4'h1:    HEX0 <= SEG_1;
407      4'h2:    HEX0 <= SEG_2;
408      4'h3:    HEX0 <= SEG_3;
409      4'h4:    HEX0 <= SEG_4;
410      4'h5:    HEX0 <= SEG_5;
411      4'h6:    HEX0 <= SEG_6;
412      4'h7:    HEX0 <= SEG_7;
413      4'h8:    HEX0 <= SEG_8;
414      4'h9:    HEX0 <= SEG_9;
415      4'hA:    HEX0 <= SEG_A;
416      4'hB:    HEX0 <= SEG_B;
417      4'hC:    HEX0 <= SEG_C;

```

```

406         4'h1:  HEX0 <= SEG_1;
407         4'h2:  HEX0 <= SEG_2;
408         4'h3:  HEX0 <= SEG_3;
409         4'h4:  HEX0 <= SEG_4;
410         4'h5:  HEX0 <= SEG_5;
411         4'h6:  HEX0 <= SEG_6;
412         4'h7:  HEX0 <= SEG_7;
413         4'h8:  HEX0 <= SEG_8;
414         4'h9:  HEX0 <= SEG_9;
415         4'hA:  HEX0 <= SEG_A;
416         4'hB:  HEX0 <= SEG_B;
417         4'hC:  HEX0 <= SEG_C;
418         4'hD:  HEX0 <= SEG_D;
419         4'hE:  HEX0 <= SEG_E;
420         4'hF:  HEX0 <= SEG_F;
421         default: HEX0 <= SEG_BLANK;
422     endcase
423 end
424
425 // =====
426 // default -- safety net
427 //   Catches any invalid display_mode value
428 //   (e.g. multi-bit set, all-zero). Drives all
429 //   HEX blank.
430 // =====
431 default: begin
432     HEX0 <= SEG_BLANK;
433     HEX1 <= SEG_BLANK;
434     HEX2 <= SEG_BLANK;
435     HEX3 <= SEG_BLANK;
436     HEX4 <= SEG_BLANK;
437     HEX5 <= SEG_BLANK;
438 end
439
440 endcase
441 end
442
443 endmodule

```

RNG module

```
C:\ADD_DEMOS\main project\rng\lfsr_rng_rev1.v - Default *
Ln#
9 ***** Functionality Description *****
10 16-bit LFSR random number generator.
11
12 - LFSR runs continuously (advances every clock cycle).
13 - Power-up state is 16'hFFFF via initial block (no system input
14   required to start).
15 - Optional active-low rst forces state back to 16'hFFFF.
16 - A single 'pulse' input captures the current LFSR state into the
17   output register, where it is held until the next pulse.
18 - Maximal-length polynomial:  $x^{16} + x^{14} + x^{12} + x^{11} + 1$ 
19   (taps on bits 15, 13, 12, 10), 65535-state cycle.
20 *****
21 */
22 module lfsr_rng(clk, rst, pulse, rng_out);
23
24   input clk, rst, pulse;
25   output reg [15:0] rng_out;
26
27   parameter HIGH = 1'b1;
28   parameter LOW = 1'b0;
29   parameter SEED = 16'hFFFF;
30
31   reg [15:0] lfsr_reg;
32
33   // Power-up initialization (no input required from system)
34   initial begin
35     lfsr_reg = SEED;
36     rng_out = SEED;
37   end
38
39   always @(posedge clk) begin
40     if (rst == LOW) begin
41       lfsr_reg <= SEED;
42       rng_out <= SEED;
43     end
44     else begin
45       // LFSR shifts every cycle -- continuously running
46       lfsr_reg <= (lfsr_reg[14:0],
47                  lfsr_reg[15] ^ lfsr_reg[13] ^
48                  lfsr_reg[12] ^ lfsr_reg[10]);
49
50       // Output captures the LFSR state on pulse and holds
51       // it until the next pulse arrives.
52       if (pulse == HIGH) begin
53         rng_out <= lfsr_reg;
54       end
55     end
56   end
57 endmodule
```

Score Tracker

```
C:\ADD_DEMOS\main project\score tracker\score_unit_rev1.v (/tb_score_unit/DUT_SCORE) - Default *
Ln#
21 Personal high, leaderboard, and per-player storage are handled
22 externally in ScoreRAM, which simply reads 'score' from this
23 module.
24 *****
25 */
26 module score_unit(clk, rst, soft_rst, pulse, multiplier, score);
27
28   input clk, rst, soft_rst, pulse;
29   input [1:0] multiplier;
30   output reg [7:0] score;
31
32   parameter HIGH = 1'b1;
33   parameter LOW = 1'b0;
34
35   initial begin
36     score = 8'd0;
37   end
38
39   always @(posedge clk) begin
40     if (rst == LOW) begin
41       // Hard reset from system
42       score <= 8'd0;
43     end
44     else if (soft_rst == HIGH) begin
45       // Internal soft reset (e.g. start of a new round)
46       score <= 8'd0;
47     end
48     else if (pulse == HIGH) begin
49       // Saturate at 255 to prevent rollover.
50       // Width-extend the threshold so the compare doesn't
51       // truncate to 8 bits and miss the overflow case.
52       if (score > (8'd255 - (6'b0, multiplier)))
53         score <= 8'd255;
54       else
55         score <= score + multiplier;
56     end
57   end
58 endmodule
```

gameControl Module

```
gameControl.v
64 =====
65 */
66 module gameControl(clk, rst, timer_done, id_log_inout, game_start_key, game_control_key,
67                   verified, id_passed, difficulty_in,
68                   timer_enable, timer_reconfig, enable_mega_compar,
69                   pulse_w_ram, pulse_scrl_ram,
70                   difficulty_out, logout_req, score_soft_rst,
71                   display_mode);
72
73 // ----- System -----
74 input clk, rst;
75
76 // ----- Inputs from twoDigitTimer -----
77 input timer_done;
78
79 // ----- Inputs from raw KEYS / switches -----
80 input id_log_inout; // active-LOW: LOW = logout pressed
81 input game_start_key; // shaped pulse; HIGH = "start the round"
82 input game_control_key; // shaped pulse; HIGH = "scroll" / "restart"
83
84 // ----- Inputs from Authentication -----
85 input verified; // HIGH after successful PW (LoggedIn)
86 input id_passed; // HIGH after ID match, before PW pass (NEW)
87
88 // ----- Inputs from difficulty switches -----
89 input [1:0] difficulty_in;
90
91 // ----- Outputs to other modules -----
92 output reg timer_enable, timer_reconfig;
93 output reg enable_mega_compar;
94 output reg pulse_w_ram, pulse_scrl_ram;
95 output reg [1:0] difficulty_out;
96 output reg logout_req;
97 output reg score_soft_rst;
98
99 // ----- One-hot mode output (NEW) -----
100 output reg [6:0] display_mode;
101
102 // =====
103 // State encoding
104 // Existing state numbers preserved (UNAUTH=1 .. LBOARD=6);
105 // ID_IN tucked in at 7 to avoid renumbering existing tests.
106 // =====
107 parameter UNAUTHENTICATED = 4'd1;
108 parameter AUTHENTICATED = 4'd2;
109 parameter WAIT_FOR_START = 4'd3;
110 parameter GAME_RUNNING = 4'd4;
111 parameter END_GAME = 4'd5; // renamed from GAME_OVER
112 parameter LBOARD = 4'd6; // renamed from WAIT_TO_RESTART
113 parameter ID_IN = 4'd7; // NEW state
114
115 // ----- Logic-level aliases -----
116 parameter HIGH = 1'b1;
```

```

gameControl.v
115 // ----- Logic-level aliases -----
116 parameter HIGH = 1'b1;
117 parameter LOW = 1'b0;
118
119 // ----- Difficulty levels -----
120 parameter NOT_SELECTED = 2'b00;
121 parameter EASY = 2'b01;
122 parameter MEDIUM = 2'b10;
123 parameter HARD = 2'b11;
124
125 // ----- One-hot display_mode patterns -----
126 parameter MODE_UNAUTHED = 7'b000_0001;
127 parameter MODE_ID_IN = 7'b000_0010;
128 parameter MODE_AUTHED = 7'b000_0100;
129 parameter MODE_PRE_GAME = 7'b000_1000;
130 parameter MODE_PLAYING = 7'b001_0000;
131 parameter MODE_END_GAME = 7'b010_0000;
132 parameter MODE_LBOARD = 7'b100_0000;
133
134 // =====
135 // END_GAME hold counter
136 // 50,000,000 cycles at 50 MHz = 1.0 s. Sim-speed value
137 // commented for quick simulation runs.
138 // =====
139 //parameter END_GAME_HOLD = 26'd3; // sim-speed (4 cycles)
140 parameter END_GAME_HOLD = 26'd49_999_999; // production: 1 s @ 50 MHz
141
142 reg [25:0] end_game_counter;
143
144 // ----- State register -----
145 reg [3:0] state = UNAUTHENTICATED;
146 reg [1:0] game_difficulty = EASY;
147
148 // =====
149 // Single-always-block FSM
150 // =====
151 always @(posedge clk) begin
152 // ----- Active-low hard reset -----
153 if (rst == LOW) begin
154 state <= UNAUTHENTICATED;
155 timer_enable <= LOW;
156 timer_reconfig <= LOW;
157 enable_mega_compar <= LOW;
158 pulse_w_ram <= LOW;
159 pulse_scr1_ram <= LOW;
160 difficulty_out <= NOT_SELECTED;
161 game_difficulty <= EASY;
162 logout_req <= LOW;
163 score_soft_rst <= HIGH;
164 display_mode <= MODE_UNAUTHED;
165 end_game_counter <= 26'd0;
166 end
167 else begin

```

gameControl.v

```
166 end
167 else begin
168     case (state)
169
170         // =====
171         // UNAUTHENTICATED -- waiting for ID match
172         // =====
173         UNAUTHENTICATED: begin
174             timer_enable      <= LOW;
175             timer_reconfig    <= LOW;
176             enable_mega_compar <= LOW;
177             pulse_w_ram       <= LOW;
178             pulse_scrl_ram    <= LOW;
179             difficulty_out     <= NOT_SELECTED;
180             logout_req        <= LOW;
181             score_soft_rst     <= HIGH;
182             display_mode       <= MODE_UNAUTHED;
183             end_game_counter   <= 26'd0;
184
185             // next state: ID got accepted by Authentication
186             if (id_passed == HIGH)
187                 state <= ID_IN;
188             else
189                 state <= UNAUTHENTICATED;
190         end
191
192         // =====
193         // ID_IN -- ID accepted, entering password
194         // Outputs match UNAUTHENTICATED almost exactly
195         // (still pre-auth, pre-game). Only display_mode
196         // changes so the LED moves.
197         // =====
198         ID_IN: begin
199             timer_enable      <= LOW;
200             timer_reconfig    <= LOW;
201             enable_mega_compar <= LOW;
202             pulse_w_ram       <= LOW;
203             pulse_scrl_ram    <= LOW;
204             difficulty_out     <= NOT_SELECTED;
205             logout_req        <= LOW;
206             score_soft_rst     <= HIGH;
207             display_mode       <= MODE_ID_IN;
208             end_game_counter   <= 26'd0;
209
210             // next state
211             if (verified == HIGH) begin
212                 // PW just passed -- enter AUTHENTICATED with
213                 // the one-cycle timer reload.
214                 state <= AUTHENTICATED;
215                 timer_reconfig <= HIGH;
216             end
217             else if (id_passed == LOW) begin
218                 // Authentication reset back to ID phase
```


gameControl.v

```
217     else if (id_passed == LOW) begin
218         // Authentication reset back to ID phase
219         // (4-strike fail or other internal reset)
220         state <= UNAUTHENTICATED;
221     end
222     else begin
223         state <= ID_IN;
224     end
225 end
226
227 // =====
228 // AUTHENTICATED -- one-cycle timer reload
229 // =====
230 AUTHENTICATED: begin
231     timer_enable          <= LOW;
232     timer_reconfig        <= LOW; // drop pulse after 1 cycle
233     enable_mega_compar     <= LOW;
234     pulse_w_ram           <= LOW;
235     pulse_sctl_ram        <= LOW;
236     difficulty_out         <= NOT_SELECTED;
237     logout_req            <= LOW;
238     score_soft_rst        <= HIGH;
239     display_mode          <= MODE_AUTHED;
240     end_game_counter      <= 26'd0;
241
242     // next state
243     if (verified == LOW)
244         state <= UNAUTHENTICATED; // logout cascade fired
245     else
246         state <= WAIT_FOR_START;
247 end
248
249 // =====
250 // WAIT_FOR_START -- difficulty selection,
251 // awaiting start key
252 // =====
253 WAIT_FOR_START: begin
254     timer_enable          <= LOW;
255     timer_reconfig        <= LOW;
256     enable_mega_compar     <= LOW;
257     pulse_w_ram           <= LOW;
258     pulse_sctl_ram        <= LOW;
259     logout_req            <= LOW;
260     score_soft_rst        <= HIGH;
261     display_mode          <= MODE_PRE_GAME;
262     end_game_counter      <= 26'd0;
263
264     // latch difficulty from switches
265     if (difficulty_in != NOT_SELECTED) begin
266         game_difficulty <= difficulty_in;
267         difficulty_out  <= difficulty_in;
268     end
269     else begin
```

```

gameControl.v
268     end
269   else begin
270     difficulty_out <= game_difficulty;
271   end
272
273   // next state
274   if (id_log_inout == HIGH) begin
275     // logout pressed
276     state <= UNAUTHENTICATED;
277     logout_req <= HIGH;
278   end
279   else if (game_start_key == HIGH && difficulty_in != NOT_SELECTED) begin
280     state <= GAME_RUNNING;
281   end
282   else begin
283     state <= WAIT_FOR_START;
284   end
285 end
286
287 // =====
288 // GAME_RUNNING -- gameplay active
289 // =====
290 GAME_RUNNING: begin
291   timer_enable <= HIGH;
292   timer_reconfig <= LOW;
293   enable_mega_compar <= HIGH;
294   pulse_w_ram <= LOW;
295   pulse_scri_ram <= LOW;
296   difficulty_out <= game_difficulty;
297   logout_req <= LOW;
298   score_soft_rst <= LOW;
299   display_mode <= MODE_PLAYING;
300   end_game_counter <= 26'd0;
301
302   // next state
303   if (timer_done == HIGH) begin
304     // round timer hit zero -> end of game
305     state <= END_GAME;
306     timer_enable <= LOW;
307     enable_mega_compar <= LOW;
308     pulse_w_ram <= HIGH; // one-cycle write pulse
309   end
310   else if (verified == LOW) begin
311     // logout cascade fired during play
312     state <= UNAUTHENTICATED;
313   end
314   else begin
315     state <= GAME_RUNNING;
316   end
317 end
318
319 // =====
320 // END_GAME -- 1-second hold before LBOARD

```

```

319 // =====
320 // END_GAME -- 1-second hold before LBOARD
321 // pulse_w_ram was raised on entry from
322 // GAME_RUNNING; drop it back to LOW now so
323 // ScoreRAM sees a single-cycle pulse.
324 // =====
325 END_GAME: begin
326     timer_enable         <= LOW;
327     timer_reconfig       <= LOW;
328     enable_mega_compar   <= LOW;
329     pulse_w_ram          <= LOW; // trailing edge of write pulse
330     pulse_scl_ram        <= LOW;
331     difficulty_out        <= game_difficulty;
332     logout_req           <= LOW;
333     score_soft_rst       <= LOW; // preserve final score
334     display_mode         <= MODE_END_GAME;
335
336     // hold counter -> LBOARD when it expires
337     if (end_game_counter == END_GAME_HOLD) begin
338         state          <= LBOARD;
339         end_game_counter <= 26'd0;
340     end
341     else begin
342         end_game_counter <= end_game_counter + 1;
343     end
344 end
345
346 // =====
347 // LBOARD -- leaderboard view
348 // game_control_key = scroll -> stay, pulse scroll
349 // game_start_key   = restart -> WAIT_FOR_START
350 // id_log_inout HIGH = logout -> UNAUTHENTICATED
351 // =====
352 LBOARD: begin
353     timer_enable         <= LOW;
354     timer_reconfig       <= LOW;
355     enable_mega_compar   <= LOW;
356     pulse_w_ram          <= LOW;
357     pulse_scl_ram        <= LOW;
358     difficulty_out        <= game_difficulty;
359     logout_req           <= LOW;
360     score_soft_rst       <= LOW; // preserve scores for display
361     display_mode         <= MODE_LBOARD;
362     end_game_counter     <= 26'd0;
363
364     // next state
365     if (id_log_inout == HIGH) begin
366         // logout pressed -> back to UNAUTHENTICATED
367         state          <= UNAUTHENTICATED;
368         logout_req     <= HIGH;
369     end
370     else if (game_start_key == HIGH) begin
371         // restart with same difficulty -> back to PRE GAME

```

```

gameControl.v
364 // next state
365 if (id_log_inout == HIGH) begin
366     // logout pressed -> back to UNAUTHENTICATED
367     state      <= UNAUTHENTICATED;
368     logout_req <= HIGH;
369 end
370 else if (game_start_key == HIGH) begin
371     // restart with same difficulty -> back to PRE_GAME
372     state      <= WAIT_FOR_START;
373     timer_reconfig <= HIGH; // reload timer to 99
374 end
375 else if (game_control_key == HIGH) begin
376     // scroll to next leaderboard entry
377     state      <= LBOARD;
378     pulse_scrl_ram <= HIGH; // one-cycle pulse
379 end
380 else begin
381     state <= LBOARD;
382 end
383 end
384
385 // =====
386 // default -- safety net
387 // =====
388 default: begin
389     state      <= UNAUTHENTICATED;
390     timer_enable <= LOW;
391     timer_reconfig <= LOW;
392     enable_mega_compar <= LOW;
393     pulse_w_ram <= LOW;
394     pulse_scrl_ram <= LOW;
395     difficulty_out <= NOT_SELECTED;
396     logout_req <= LOW;
397     score_soft_rst <= HIGH;
398     display_mode <= MODE_UNAUTHED;
399     end_game_counter <= 26'd0;
400 end
401
402 endcase
403 end
404
405 endmodule
406
407

```

Game Timer Module

```
GameTimer.v
1  module GameTimer (rst, clk, Enable, soft_rst, GameDifficulty, TimeOut);
2      input rst, clk;
3      input Enable, soft_rst;
4      input [1:0] GameDifficulty;
5
6      output TimeOut;
7
8      reg TimeOut;
9      reg [4:0] count;
10     reg [4:0] countTerminal;
11
12     wire OneMSSignal, HundredOut;
13
14     LFSR OneMSLFSR (clk, rst, soft_rst, Enable, OneMSSignal);
15     CountTo100 HundredCount ( OneMSSignal, rst, soft_rst, clk, HundredOut);
16
17
18     always @(posedge clk)
19     begin
20         if (rst == 1'b0)
21         begin
22             TimeOut <= 1'b0;
23             count <= 5'b000000;
24         end
25         else
26         begin
27             if (soft_rst == 1'b1)
28             count <= 5'b000000;
29
30             TimeOut <= 1'b0;
31             case (GameDifficulty)
32             2'b01: begin
33                 //countTerminal <= 5'b00011; // commented out
34                 countTerminal <= 5'b10100;
35             end
36
37             2'b10: begin
38                 //countTerminal <= 5'b00010; // commented out
39                 countTerminal <= 5'b01111;
40             end
41
42             2'b11: begin
43                 //countTerminal <= 5'b00001; // commented out
44                 countTerminal <= 5'b01010;
45             end
46
47             default: begin
48                 countTerminal <= 5'b10100;
49             end
50             endcase
51
52             if (HundredOut == 1'b1)
53             begin
54                 count <= count + 1;
55                 if (count == countTerminal) begin
56                     TimeOut <= 1'b1;
57                     count <= 5'b000000;
58                 end
59             end
60         end
61     end
62 end
63
64 endmodule
65
```

```

1
2
3 // Edited by Z. Mir (ZM): added NumberID_to_PW [15:0] output.
4 // Pass-through wrapper extension -- ID_check now exposes the
5 // user-typed 16-bit ID, ID_Auth just propagates it up.
6
7 module ID_Auth(Digit, B_in_shaped, Logout_Req_from_PW, clk, reset, matchedID, ID_passed, PlayerID_to_PW, NumberID_to_PW);
8     input [3:0] Digit;
9     input B_in_shaped, Logout_Req_from_PW, clk, reset;
10
11     output matchedID, ID_passed;
12     output [4:0] PlayerID_to_PW;
13     output [15:0] NumberID_to_PW; // ZM: new
14
15     wire [15:0] ID_ROMdata;
16     wire [4:0] ID_ROMAddr;
17
18     ID_check ID_check_top(Digit, B_in_shaped, Logout_Req_from_PW,
19     ID_ROMdata, clk, reset, matchedID, PlayerID_to_PW, ID_ROMAddr, NumberID_to_PW); // ZM: added NumberID_to_PW
20
21     PlayerID_ROM PlayerID_ROM_top(ID_ROMAddr, clk, ID_ROMdata);
22
23     assign ID_passed = matchedID;
24
25 endmodule

```

ID_Check

```

1
2
3 // Edited by Z. Mir (ZM): added NumberID_to_PW [15:0] output.
4 // Carries the user-typed 16-bit ID number out of ID_check so
5 // the PW chain can forward it to scoreRAM. Latched from
6 // EnterID_reg on entry to IDPASSED, cleared on logout/reset.
7 // Port appended at end of list to keep existing positional
8 // instantiations source-compatible.
9
10 module ID_check(Digit, B_in_shaped, Logout_Req_from_PW, ID_ROMdata, clk, reset, matchedID, PlayerID_to_PW, ID_ROMAddr, NumberID_to_PW);
11     input [3:0] Digit;
12     input B_in_shaped, Logout_Req_from_PW, clk, reset;
13     input [15:0] ID_ROMdata;
14
15     output reg matchedID;
16     output reg [4:0] PlayerID_to_PW, ID_ROMAddr;
17     output reg [15:0] NumberID_to_PW; // ZM: new
18
19     parameter DIGIT1 = 0, DIGIT2 = 1, DIGIT3 = 2, DIGIT4 = 3, FETCHROM = 4, ROMWAIT1 = 5;
20     parameter ROMWAIT2 = 6, CATCHROM = 7, COMPARE = 8, IDPASSED = 9, IDSTATUS = 10;
21
22     reg [3:0] State;
23     reg [15:0] EnterID_reg;
24     reg [15:0] ID_ROMdata_reg;
25
26     always @(posedge clk) begin
27         if (reset == 1'b0) begin
28             matchedID <= 1'b0;
29             PlayerID_to_PW <= 5'd0;
30             ID_ROMAddr <= 5'd0;
31             EnterID_reg <= 16'd0;
32             ID_ROMdata_reg <= 16'd0;
33             NumberID_to_PW <= 16'd0; // ZM: new
34             State <= DIGIT1; end
35
36     else begin
37         case(State)
38             DIGIT1: begin
39                 matchedID <= 1'b0;
40                 PlayerID_to_PW <= 5'd0;
41                 ID_ROMAddr <= 5'd0;
42                 EnterID_reg <= 16'd0;
43                 ID_ROMdata_reg <= 16'd0;
44
45                 if (B_in_shaped == 1'b1) begin
46                     EnterID_reg[15:12] <= Digit;
47                     State <= DIGIT2; end
48
49                 else begin
50                     State <= DIGIT1; end
51             end
52
53             DIGIT2: begin
54                 matchedID <= 1'b0;
55
56                 if (B_in_shaped == 1'b1) begin
57

```

LFSR

```
LFSR.v
1  module LFSR(clk, rst, soft_rst, Enable, TimeOut);
2      input clk, rst;
3      input Enable, soft_rst;
4      output TimeOut;
5      reg TimeOut;
6      reg [15:0] LFSR;
7      wire feedback = LFSR[15];
8
9      always @(posedge clk)
10     begin
11         if (rst == 1'b0)
12             begin
13                 LFSR <= 16'hFFFF;
14                 TimeOut <= 1'b0;
15             end
16         else
17             begin
18                 TimeOut <= 1'b0;
19                 if (soft_rst == 1'b1) begin
20                     LFSR <= 16'hFFFF;
21                     TimeOut <= 1'b0;
22                 end
23                 else if (Enable == 1'b1)
24                     begin
25                         LFSR[0] <= feedback;
26                         LFSR[1] <= LFSR[0];
27                         LFSR[2] <= LFSR[1] ^ feedback;
28                         LFSR[3] <= LFSR[2] ^ feedback;
29                         LFSR[4] <= LFSR[3];
30                         LFSR[5] <= LFSR[4] ^ feedback;
31                         LFSR[6] <= LFSR[5];
32                         LFSR[7] <= LFSR[6];
33                         LFSR[8] <= LFSR[7];
34                         LFSR[9] <= LFSR[8];
35                         LFSR[10] <= LFSR[9];
36                         LFSR[11] <= LFSR[10];
37                         LFSR[12] <= LFSR[11];
38                         LFSR[13] <= LFSR[12];
39                         LFSR[14] <= LFSR[13];
40                         LFSR[15] <= LFSR[14];
41                         //if (LFSR == 16'hff3b) // commented out
42                         if ( LFSR == 16'h6db6)
43                             begin
44                                 LFSR <= 16'hFFFF;
45                                 TimeOut <= 1'b1;
46                             end
47                         end
48                     end
49             end
50     end
endmodule
```

LFSR_rng_rev1

```
lfsr_rng_rev1.v
22 module lfsr_rng(clk, rst, pulse, rng_out);
23
24     input  clk, rst, pulse;
25     output reg [15:0] rng_out;
26
27     parameter HIGH = 1'b1;
28     parameter LOW  = 1'b0;
29     parameter SEED = 16'hFFFF;
30
31     reg [15:0] lfsr_reg;
32
33     // Power-up initialization (no input required from system)
34     initial begin
35         lfsr_reg = SEED;
36         rng_out  = SEED;
37     end
38
39     always @(posedge clk) begin
40         if (rst == LOW) begin
41             lfsr_reg <= SEED;
42             rng_out  <= SEED;
43         end
44         else begin
45             // LFSR shifts every cycle -- continuously running
46             lfsr_reg <= {lfsr_reg[14:0],
47                         lfsr_reg[15] ^ lfsr_reg[13] ^
48                         lfsr_reg[12] ^ lfsr_reg[10]};
49
50             // Output captures the LFSR state on pulse and holds
51             // it until the next pulse arrives.
52             if (pulse == HIGH) begin
53                 rng_out <= lfsr_reg;
54             end
55         end
56     end
57
58 endmodule
```

LFSRTimer


```
LFSRTimer.v
16     output reg timeout;
17
18     reg [15:0] LFSR;
19     wire feedback = LFSR[15];
20
21     parameter TERMEND = 16'h1C51;
22
23     always @(posedge clk) begin
24         if(rst == 1'b0) begin
25             LFSR[15:0] <= 16'hFFFF;
26             timeout <= 1'b0;
27         end else begin
28             if (enable == 1) begin // only counting if the enable is one
29                 if(LFSR == TERMEND) begin
30                     timeout <= 1'b1;
31                     LFSR[15:0] <= 16'hFFFF;
32                 end else begin
33                     LFSR[0] <= feedback;
34                     LFSR[1] <= LFSR[0];
35                     LFSR[2] <= LFSR[1] ^ feedback;
36                     LFSR[3] <= LFSR[2];
37                     LFSR[4] <= LFSR[3];
38                     LFSR[5] <= LFSR[4] ^ feedback;
39                     LFSR[6] <= LFSR[5] ^ feedback;
40                     LFSR[7] <= LFSR[6];
41                     LFSR[8] <= LFSR[7];
42                     LFSR[9] <= LFSR[8];
43                     LFSR[10] <= LFSR[9];
44                     LFSR[11] <= LFSR[10];
45                     LFSR[12] <= LFSR[11];
46                     LFSR[13] <= LFSR[12];
47                     LFSR[14] <= LFSR[13];
48                     LFSR[15] <= LFSR[14];
49                     timeout <= 1'b0;
50                 end
51             end else begin // incrase the count with clock edge if
52                 LFSR[15:0] <= LFSR[15:0];
53                 timeout <= 0;
54             end
55         end
56     end
57
58 endmodule
```

MegaComparator

```

61  =====
62  */
63  module megaComparator(
64      clk, rst,
65      enable, difficulty, score_soft_rst,
66      player_in, player_load_in,
67      score, display_random
68  );
69      // ---- common ----
70      input clk, rst;
71
72      // ---- from gameControl ----
73      input enable;
74      input [1:0] difficulty;
75      input score_soft_rst;
76
77      // ---- from external pins ----
78      input [3:0] player_in;
79      input player_load_in;
80
81      // ---- outputs ----
82      output [7:0] score;
83      output [3:0] display_random;
84
85      // ---- internal pulse: Comparator -> score_unit ----
86      wire score_pulse;
87
88      // ---- Comparator: round FSM (RNG + GameTimer embedded) ----
89      Comparator u_comparator(
90          .rst (rst),
91          .clk (clk),
92          .Enable (enable),
93          .GameDifficulty(difficulty),
94          .Player_in (player_in),
95          .PlayerLoad_in (player_load_in),
96          .ScorePulse (score_pulse),
97          .DisplayRandom (display_random)
98      );
99
100      // ---- score_unit: saturating accumulator ----
101      score_unit u_score_unit(
102          .clk (clk),
103          .rst (rst),
104          .soft_rst (score_soft_rst),
105          .pulse (score_pulse),
106          .multiplier(difficulty),
107          .score (score)
108      );
109
110  endmodule

```

OneSecTimerLFSR

```
oneSecTimerLFSR.v
13
14 module oneSecTimerLFSR(clk, rst, enable, timeout);
15     input clk, rst, enable;
16     output timeout;
17
18     wire t_1ms;    // timeout from coutnTo4
19     wire t_100ms;  // timeout from countTo3
20
21     // LFSRTimer(clk, rst, enable, timeout);
22     LFSRTimer u_LFSRTimer(clk, rst, enable, t_1ms);
23
24     // t_1ms enables the contTO3 and it start to count when
25     countTo3 u_countTo3(clk, rst, t_1ms, t_100ms);
26
27     // t_100ms enables the contTO3 and it start to count when
28     countTo2 u_countTo2(clk, rst, t_100ms, timeout);
29
30 endmodule
```

Password_ROM

```

40 module Password_ROM (
41     address,
42     clock,
43     q);
44
45     input  [4:0] address;
46     input    clock;
47     output [15:0] q;
48 `ifndef ALTERA_RESERVED_QIS
49 // synopsys translate_off
50 `endif
51     tri1    clock;
52 `ifndef ALTERA_RESERVED_QIS
53 // synopsys translate_on
54 `endif
55
56     wire [15:0] sub_wire0;
57     wire [15:0] q = sub_wire0[15:0];
58
59     altsyncram altsyncram_component (
60         .address_a (address),
61         .clock0 (clock),
62         .q_a (sub_wire0),
63         .aclr0 (1'b0),
64         .aclr1 (1'b0),
65         .address_b (1'b1),
66         .addressstall_a (1'b0),
67         .addressstall_b (1'b0),
68         .byteena_a (1'b1),
69         .byteena_b (1'b1),
70         .clock1 (1'b1),
71         .clocken0 (1'b1),
72         .clocken1 (1'b1),
73         .clocken2 (1'b1),
74         .clocken3 (1'b1),
75         .data_a ({16{1'b1}}),
76         .data_b (1'b1),
77         .eccstatus (),
78         .q_b (),
79         .rden_a (1'b1),
80         .rden_b (1'b1),
81         .wren_a (1'b0),
82         .wren_b (1'b0));
83
84     defparam
85         altsyncram_component.address_aclr_a = "NONE",
86         altsyncram_component.clock_enable_input_a = "BYPASS",
87         altsyncram_component.clock_enable_output_a = "BYPASS",
88         altsyncram_component.init_file = "Password_ROM.rif"
89     `else
90         altsyncram_component.init_file = "Password_ROM.hex"
91     `endif
92 ,
93     altsyncram_component.intended_device_family = "Cyclone V",
94     altsyncram_component.lpm_hint = "ENABLE_RUNTIME_MOD=NO",
95     altsyncram_component.lpm_type = "altsyncram",
96     altsyncram_component.numwords_a = 32,
97     altsyncram_component.operation_mode = "ROM",
98     altsyncram_component.outdata_aclr_a = "NONE",
99     altsyncram_component.outdata_reg_a = "CLOCK0",
100     altsyncram_component.widthad_a = 5,
101     altsyncram_component.width_a = 16,
102     altsyncram_component.width_byteena_a = 1;
103
104 endmodule
105

```

PlayerID_ROM

```
PlayerID_ROM.v
40 module PlayerID_ROM (
41     address,
42     clock,
43     q);
44
45     input  [4:0] address;
46     input    clock;
47     output [15:0] q;
48
49     `ifndef ALTERA_RESERVED_QIS
50     // synopsys translate_off
51     `endif
52     tri1    clock;
53     `ifndef ALTERA_RESERVED_QIS
54     // synopsys translate_on
55     `endif
56
57     wire [15:0] sub_wire0;
58     wire [15:0] q = sub_wire0[15:0];
59
60     altsyncram altsyncram_component (
61         .address_a (address),
62         .clock0 (clock),
63         .q_a (sub_wire0),
64         .aclr0 (1'b0),
65         .aclr1 (1'b0),
66         .address_b (1'b1),
67         .addressstall_a (1'b0),
68         .addressstall_b (1'b0),
69         .byteena_a (1'b1),
70         .byteena_b (1'b1),
71         .clock1 (1'b1),
72         .clocken0 (1'b1),
73         .clocken1 (1'b1),
74         .clocken2 (1'b1),
75         .clocken3 (1'b1),
76         .data_a ({16{1'b1}}),
77         .data_b (1'b1),
78         .eccstatus (),
79         .q_b (),
80         .rden_a (1'b1),
81         .rden_b (1'b1),
82         .wren_a (1'b0),
83         .wren_b (1'b0));
84
85     defparam
86         altsyncram_component.address_aclr_a = "NONE",
87         altsyncram_component.clock_enable_input_a = "BYPASS",
88         altsyncram_component.clock_enable_output_a = "BYPASS",
89
90     `ifdef NO_PLI
91         altsyncram_component.init_file = "PlayerID_ROM.rif"
92     `else
93         altsyncram_component.init_file = "PlayerID_ROM.hex"
94     `endif
95
96     ,
97     altsyncram_component.intended_device_family = "Cyclone V",
98     altsyncram_component.lpm_hint = "ENABLE_RUNTIME_MOD=NO",
99     altsyncram_component.lpm_type = "altsyncram",
100     altsyncram_component.numwords_a = 32,
101     altsyncram_component.operation_mode = "ROM",
102     altsyncram_component.outdata_aclr_a = "NONE",
103     altsyncram_component.outdata_reg_a = "CLOCK0",
104     altsyncram_component.widthad_a = 5,
105     altsyncram_component.width_a = 16,
106     altsyncram_component.width_byteena_a = 1;
107
108 endmodule
```

PW_Auth

```

PW_Auth.v
1  // C:\Users\bryan\Downloads\Fnal_project_v3_with_bug (1)\src\PlayerID_ROM.v
2  // Edited by Z. Mir (ZM): added NumberID_from_ID_Auth [15:0]
3  // input and NumberID_to_scoreRAM [15:0] output. PW_Auth now
4  // passes the user-typed 16-bit ID from ID_Auth down to
5  // PW_check (which latches it on PWPASSED) and back out to
6  // scoreRAM. Port positions match Authentication.v's positional
7  // instantiation order (NumberID interspersed with PlayerID).
8  // =====
9
10 module PW_Auth(matchedID, Digit, B_in_shaped, PlayerID_from_ID_Auth, NumberID_from_ID_Auth,
11 Logout_Req_from_GameCtrl, clk, reset, LoggedIn, LoggedOut, PlayerID_to_scoreRAM, NumberID_to_scoreRAM, Logout_Req_to_ID_check);
12
13 input matchedID, B_in_shaped, Logout_Req_from_GameCtrl, clk, reset;
14 input [3:0] Digit;
15 input [4:0] PlayerID_from_ID_Auth;
16 input [15:0] NumberID_from_ID_Auth; // ZM: new
17
18 output LoggedIn, LoggedOut, Logout_Req_to_ID_check;
19 output [4:0] PlayerID_to_scoreRAM;
20 output [15:0] NumberID_to_scoreRAM; // ZM: new
21
22 wire [4:0] PW_ROMAddr;
23 wire [15:0] PW_ROMdigit;
24
25 Password_ROM Password_ROM_top (PW_ROMAddr, clk, PW_ROMdigit);
26 PW_check PW_check_top (matchedID, Digit, B_in_shaped, PW_ROMdigit, PlayerID_from_ID_Auth,
27 NumberID_from_ID_Auth, Logout_Req_from_GameCtrl, clk, reset, PW_ROMAddr, LoggedIn, LoggedOut,
28 PlayerID_to_scoreRAM, NumberID_to_scoreRAM, Logout_Req_to_ID_check); // ZM: added NumberID in/out
29
30 endmodule

```

PW_check

```

PW_check.v
9 module PW_check(matchedID, Digit, B_in_shaped, PW_ROMdigit, PlayerID_from_ID_Auth, NumberID_from_ID_Auth,
10 Logout_Req_from_GameCtrl, clk, reset, PW_ROMAddr, LoggedIn, LoggedOut, PlayerID_to_scoreRAM, NumberID_to_scoreRAM, Logout_Req_to_ID_check);
11 input matchedID, B_in_shaped, Logout_Req_from_GameCtrl, clk, reset;
12 input [3:0] Digit;
13 input [15:0] PW_ROMdigit;
14 input [4:0] PlayerID_from_ID_Auth;
15 input [15:0] NumberID_from_ID_Auth; // ZM: new
16
17 output reg LoggedIn, LoggedOut, Logout_Req_to_ID_check;
18 output reg [4:0] PW_ROMAddr;
19 output reg [4:0] PlayerID_to_scoreRAM;
20 output reg [15:0] NumberID_to_scoreRAM; // ZM: new
21
22 reg [15:0] EnterPW_reg;
23 reg [15:0] PW_ROMdata_reg;
24
25 reg [1:0] FailCounter;
26 reg [3:0] State;
27
28 parameter INIT = 0, DIGIT1 = 1, DIGIT2 = 2, DIGIT3 = 3, DIGIT4 = 4, FETCHROM = 5, ROMWAIT1 = 6, ROMWAIT2 = 7;
29 parameter CATCHROM = 8, COMPARE = 9, PWPASSED = 10, PWSTATUS = 11, LOGOUTWAIT = 12;
30
31 parameter HIGH = 1'b1;
32 parameter LOW = 1'b0;
33
34 always @(posedge clk) begin
35 if (reset == LOW) begin
36 LoggedIn <= LOW;
37 LoggedOut <= HIGH;
38 Logout_Req_to_ID_check <= LOW;
39 PW_ROMAddr <= 5'd0;
40 PlayerID_to_scoreRAM <= 3'd0;
41 NumberID_to_scoreRAM <= 16'd0; // ZM: new
42 EnterPW_reg <= 16'd0;
43 PW_ROMdata_reg <= 16'd0;
44 FailCounter <= 2'd0;
45 State <= INIT;
46 end
47
48 else begin
49 case(State)
50 INIT: begin
51 LoggedIn <= LOW;
52 LoggedOut <= HIGH;
53 Logout_Req_to_ID_check <= LOW;
54 PW_ROMAddr <= 5'd0;
55 PlayerID_to_scoreRAM <= 3'd0;
56 NumberID_to_scoreRAM <= 16'd0; // ZM: new
57 EnterPW_reg <= 16'd0;
58 PW_ROMdata_reg <= 16'd0;
59 FailCounter <= 2'd0;
60
61 if (matchedID == HIGH) begin
62 State <= DIGIT1;
63 end
64
65 else begin
66 State <= INIT;
67 end
68 end
69
70 DIGIT1: begin
71 LoggedIn <= LOW;
72 LoggedOut <= HIGH;
73 Logout_Req_to_ID_check <= LOW;

```

PW_check.v

```
72      LoggedOut <= HIGH;
73      Logout_Req_to_ID_check <= LOW;
74      PW_ROMAddr <= 5'd0;
75      EnterPW_reg <= 16'd0;
76      PW_ROMdata_reg <= 16'd0;
77
78      if (B_in_shaped == HIGH) begin
79          EnterPW_reg[15:12] <= Digit;
80          State <= DIGIT2;
81      end
82
83      else begin
84          State <= DIGIT1;
85      end
86  end
87
88  DIGIT2: begin
89      LoggedIn <= LOW;
90      LoggedOut <= HIGH;
91      Logout_Req_to_ID_check <= LOW;
92
93      if (B_in_shaped == HIGH) begin
94          EnterPW_reg[11:8] <= Digit;
95          State <= DIGIT3;
96      end
97
98      else begin
99          State <= DIGIT2;
100      end
101  end
102
103  DIGIT3: begin
104      LoggedIn <= LOW;
105      LoggedOut <= HIGH;
106      Logout_Req_to_ID_check <= LOW;
107
108      if (B_in_shaped == HIGH) begin
109          EnterPW_reg[7:4] <= Digit;
110          State <= DIGIT4;
111      end
112
113      else begin
114          State <= DIGIT3;
115      end
116  end
117
118  DIGIT4: begin
119      LoggedIn <= LOW;
120      LoggedOut <= HIGH;
121      Logout_Req_to_ID_check <= LOW;
122
123      if (B_in_shaped == HIGH) begin
124          EnterPW_reg[3:0] <= Digit;
125          State <= FETCHROM;
126      end
127
128      else begin
129          State <= DIGIT4;
130      end
131  end
132
133  FETCHROM: begin
134      LoggedIn <= LOW;
135      LoggedOut <= HIGH;
136      Logout_Req_to_ID_check <= LOW;
137      State <= ROMWAIT1;
```

```
PW_check.v
135   LoggedOut <= HIGH;
136   Logout_Req_to_ID_check <= LOW;
137   State <= ROMWAIT1;
138   end
139
140   ROMWAIT1: begin
141     LoggedIn <= LOW;
142     LoggedOut <= HIGH;
143     Logout_Req_to_ID_check <= LOW;
144     State <= ROMWAIT2;
145   end
146
147   ROMWAIT2: begin
148     LoggedIn <= LOW;
149     LoggedOut <= HIGH;
150     Logout_Req_to_ID_check <= LOW;
151     State <= CATCHROM;
152   end
153
154   CATCHROM: begin
155     LoggedIn <= LOW;
156     LoggedOut <= HIGH;
157     Logout_Req_to_ID_check <= LOW;
158     PW_ROMdata_reg <= PW_ROMdigit;
159     State <= COMPARE;
160   end
161
162   COMPARE: begin
163     LoggedIn <= LOW;
164     LoggedOut <= HIGH;
165
166     if (EnterPW_reg == PW_ROMdata_reg) begin
167       State <= PWPASSED;
168     end
169
170     else begin
171       if (FailCounter == 2'd3) begin
172         State <= LOGOUTWAIT;
173         Logout_Req_to_ID_check <= HIGH;
174       end
175
176       else begin
177         PW_ROMAddr <= PW_ROMAddr + 1;
178         State <= PWSTATUS;
179       end
180     end
181   end
182
183   PWSTATUS: begin
184     if (PW_ROMdata_reg == 16'hFFFF) begin
185       FailCounter <= FailCounter + 1;
186       State <= DIGIT1;
187     end
188
189     else begin
190       State <= FETCHROM;
191     end
192   end
193
194   end
195
196   PWPASSED: begin
197     LoggedIn <= HIGH;
198     LoggedOut <= LOW;
199     PlayerID_to_scoreRAM <= PlayerID_from_ID_Auth;
200     NumberID_to_scoreRAM <= NumberID_from_ID_Auth; // ZM: latch user-typed ID
```



```
PW_check.v
198 PlayerID_to_scoreRAM <= PlayerID_from_ID_Auth;
199 NumberID_to_scoreRAM <= NumberID_from_ID_Auth; // ZM: latch user-typed ID
200
201 if (Logout_Req_from_GameCtrl == LOW) begin
202     State <= PWPASSED;
203 end
204
205 else begin
206     Logout_Req_to_ID_check <= HIGH;
207     State <= LOGOUTWAIT;
208 end
209 end
210
211 LOGOUTWAIT: begin
212     Logout_Req_to_ID_check <= LOW;
213     State <= INIT;
214 end
215
216 default: begin
217     LoggedIn <= LOW;
218     LoggedOut <= HIGH;
219     Logout_Req_to_ID_check <= LOW;
220     EnterPW_reg <= 16'd0;
221     PW_ROMdata_reg <= 16'd0;
222     FailCounter <= 2'd0;
223
224     if (matchedID == HIGH) begin
225         State <= DIGIT1;
226     end
227
228     else begin
229         State <= INIT;
230     end
231 end
232
233 endcase
234 end
235 end
236 endmodule
237
```

RNG_Comparator

```
RNG_Comparator.v
4  //The comparator will use this 16 bit number to display.
5
6  module RNG_Comparator (rst, clk, Sample_req, RandomNums);
7      input rst, clk;
8      input Sample_req;
9      output [15:0] RandomNums;
10     reg [15:0] RandomNums;
11
12     reg [15:0] LFSR;
13     wire feedback = LFSR[15];
14
15     always @(posedge clk)
16     begin
17         if (rst == 1'b0)
18         begin
19             RandomNums <= 16'h0000;
20             LFSR <= 16'hFFFF;
21         end
22         else
23         begin
24             LFSR[0] <= feedback;
25             LFSR[1] <= LFSR[0];
26             LFSR[2] <= LFSR[1] ^ feedback;
27             LFSR[3] <= LFSR[2] ^ feedback;
28             LFSR[4] <= LFSR[3];
29             LFSR[5] <= LFSR[4] ^ feedback;
30             LFSR[6] <= LFSR[5];
31             LFSR[7] <= LFSR[6];
32             LFSR[8] <= LFSR[7];
33             LFSR[9] <= LFSR[8];
34             LFSR[10] <= LFSR[9];
35             LFSR[11] <= LFSR[10];
36             LFSR[12] <= LFSR[11];
37             LFSR[13] <= LFSR[12];
38             LFSR[14] <= LFSR[13];
39             LFSR[15] <= LFSR[14];
40             if ( Sample_req == 1'b1)
41             begin
42                 RandomNums <= LFSR;
43             end
44         end
45     end
46
47 endmodule
```

score_RAM

```

40 module score_RAM (
41     address,
42     clock,
43     data,
44     wren,
45     q);
46
47     input  [4:0] address;
48     input      clock;
49     input  [23:0] data;
50     input      wren;
51     output [23:0] q;
52
53     `ifndef ALTERA_RESERVED_QIS
54     // synopsys translate_off
55     `endif
56     tri1      clock;
57
58     `ifndef ALTERA_RESERVED_QIS
59     // synopsys translate_on
60     `endif
61
62     wire [23:0] sub_wire0;
63     wire [23:0] q = sub_wire0[23:0];
64
65     altsyncram altsyncram_component (
66         .address_a (address),
67         .clock0 (clock),
68         .data_a (data),
69         .wren_a (wren),
70         .q_a (sub_wire0),
71         .aclr0 (1'b0),
72         .aclr1 (1'b0),
73         .address_b (1'b1),
74         .addressstall_a (1'b0),
75         .addressstall_b (1'b0),
76         .byteena_a (1'b1),
77         .byteena_b (1'b1),
78         .clock1 (1'b1),
79         .clocken0 (1'b1),
80         .clocken1 (1'b1),
81         .clocken2 (1'b1),
82         .clocken3 (1'b1),
83         .data_b (1'b1),
84         .eccstatus (),
85         .q_b (),
86         .rden_a (1'b1),
87         .rden_b (1'b1),
88         .wren_b (1'b0));
89
90     defparam
91         altsyncram_component.clock_enable_input_a = "BYPASS",
92         altsyncram_component.clock_enable_output_a = "BYPASS",
93         altsyncram_component.intended_device_family = "Cyclone V",
94         altsyncram_component.lpm_hint = "ENABLE_RUNTIME_MOD=NO",
95         altsyncram_component.lpm_type = "altsyncram",
96         altsyncram_component.numwords_a = 32,
97         altsyncram_component.operation_mode = "SINGLE_PORT",
98         altsyncram_component.outdata_aclr_a = "NONE",
99         altsyncram_component.outdata_reg_a = "CLOCK0",
100         altsyncram_component.power_up_uninitialized = "FALSE",
101         altsyncram_component.read_during_write_mode_port_a = "NEW_DATA_NO_NBE_READ",
102         altsyncram_component.widthad_a = 5,
103         altsyncram_component.width_a = 24,
104         altsyncram_component.width_byteena_a = 1;
105
106 endmodule

```

Score_unit

```

score_unit.v
10 Tracks the running score for the current round.
11
12 - 'pulse' input adds the current 'multiplier' value to the score.
13 - 'multiplier[1:0]' is added directly (no lookup); valid values
14   are 2'b01, 2'b10, 2'b11 (set by upstream difficulty logic).
15 - 'soft_rst' clears the score to zero without a system reset
16   (used between rounds by GameControl).
17 - Active-low 'rst' clears the score on system reset.
18 - Score saturates at 8'd255 (no rollover).
19 - All behavior is contained in a single sequential procedure.
20
21 Personal high, leaderboard, and per-player storage are handled
22 externally in ScoreRAM, which simply reads 'score' from this
23 module.
24 =====
25 */
26 module score_unit(clk, rst, soft_rst, pulse, multiplier, score);
27
28     input  clk, rst, soft_rst, pulse;
29     input  [1:0] multiplier;
30     output reg [7:0] score;
31
32     parameter HIGH = 1'b1;
33     parameter LOW  = 1'b0;
34
35     initial begin
36         score = 8'd0;
37     end
38
39     always @(posedge clk) begin
40         if (rst == LOW) begin
41             // Hard reset from system
42             score <= 8'd0;
43         end
44         else if (soft_rst == HIGH) begin
45             // Internal soft reset (e.g. start of a new round)
46             score <= 8'd0;
47         end
48         else if (pulse == HIGH) begin
49             // Saturate at 255 to prevent rollover.
50             // Width-extend the threshold so the compare doesn't
51             // truncate to 8 bits and miss the overflow case.
52             if (score > (8'd255 - {6'b0, multiplier}))
53                 score <= 8'd255;
54             else
55                 score <= score + multiplier;
56         end
57     end
58
59 endmodule

```

ScoreRAM_adapter

```
ScoreRAM_adapter.v
52  */
53  module ScoreRAM_adapter(
54      clk, rst,
55      pulse_w_ram, pulse_scri_ram,
56      my_best, global_best,
57      lboard_entry
58  );
59      // ----- system -----
60      input clk, rst;
61
62      // ----- from gameControl -----
63      input pulse_w_ram;
64      input pulse_scri_ram;
65
66      // ----- from scoreRAM_top -----
67      input [23:0] my_best;
68      input [23:0] global_best;
69
70      // ----- to DisplayCnt -----
71      output reg [23:0] lboard_entry;
72
73      // ----- logic-level aliases -----
74      parameter HIGH = 1'b1;
75      parameter LOW  = 1'b0;
76
77      // ----- view selector -----
78      // 0 = show my_best, 1 = show global_best
79      reg view_sel;
80
81      always @(posedge clk) begin
82          if (rst == LOW) begin
83              view_sel    <= 1'b0;
84              lboard_entry <= 24'h000000;
85          end
86          else begin
87              // -----
88              // View selector update.
89              // pulse_w_ram has priority over pulse_scri_ram --
90              // gameControl never asserts both in the same cycle
91              // today, but if it ever did, opening LBOARD at MyBest
92              // is the desired behavior.
93              // -----
94              if (pulse_w_ram == HIGH) begin
95                  view_sel <= 1'b0;
96              end
97              else if (pulse_scri_ram == HIGH) begin
98                  view_sel <= ~view_sel;
99              end
100
101              // -----
102              // Output mux. Reads view_sel from the start of this
103              // cycle (non-blocking semantics), so the output reflects
104              // the OLD selector value -- updated selector takes
105              // effect on the next clock. This is the standard pattern
106              // and is fine; the 1-cycle lag is invisible at 50 MHz.
107              // -----
108              if (view_sel == 1'b0)
109                  lboard_entry <= my_best;
110              else
111                  lboard_entry <= global_best;
112          end
113      end
```

scoreRam_Ctrl

```
scoreRAM_Ctrl.v
1
2 module scoreRAM_Ctrl(pulse_WRAM_from_GameCtrl, pulse_sctrl_RAM_from_GameCtrl, PlayerID_from_Auth,
3   NumberID_from_Auth, LatestScore, data_fromRAM, clk, reset, R_W_en, RAMAddr, data_toRAM, MyBest_to_DispCnt, GlobalBest_to_DispCnt);
4
5   input pulse_WRAM_from_GameCtrl, pulse_sctrl_RAM_from_GameCtrl, clk, reset;
6   input [4:0] PlayerID_from_Auth;
7   input [15:0] NumberID_from_Auth;
8   input [7:0] LatestScore;
9   input [23:0] data_fromRAM;
10
11   output reg R_W_en; // write 0, read 1
12   output reg [4:0] RAMAddr;
13   output reg [23:0] data_toRAM, MyBest_to_DispCnt, GlobalBest_to_DispCnt;
14
15   reg [7:0] LatestScore_reg;
16   reg [23:0] RAMdata_reg;
17   reg [7:0] GlobalBest_reg;
18   reg [15:0] BestPlayer_reg;
19   reg [7:0] MyBest_reg;
20   reg [3:0] State;
21
22   parameter RAMINIT = 0, WAITFORSORE = 1, FETCHRAM = 2, RAMWAIT1 = 3, RAMWAIT2 = 4;
23   parameter CATCHRAM = 5, COMPARE = 6, WRITETORAM = 7, SCOREOUTPUT = 8;
24
25   parameter HIGH = 1'b1;
26   parameter LOW = 1'b0;
27
28   always @(posedge clk) begin
29     if (reset == LOW) begin
30       R_W_en <= HIGH;
31       RAMAddr <= 5'd0;
32       data_toRAM <= 24'd0;
33       LatestScore_reg <= 8'd0;
34       GlobalBest_reg <= 8'd0;
35       BestPlayer_reg <= 16'd0; // ZM: was uninitialized, would power up as X
36       MyBest_reg <= 8'd0;
37       MyBest_to_DispCnt <= 24'd0;
38       GlobalBest_to_DispCnt <= 24'd0;
39       State <= RAMINIT;
40     end
41
42     else begin
43       case(State)
44         RAMINIT: begin
45           R_W_en <= HIGH;
46           RAMAddr <= RAMAddr + 1'b1;
47           data_toRAM <= 24'h000000; // ZM: was 0x111111 -- pre-filled every slot with bogus "Player 0x1111 scored 17"
48
49           if (RAMAddr == 5'd31) begin
50             R_W_en <= LOW;
51             State <= WAITFORSORE;
52           end
53         end
54
55         WAITFORSORE: begin
56           R_W_en <= LOW;
57
58           if (pulse_WRAM_from_GameCtrl == HIGH) begin
59             LatestScore_reg <= LatestScore;
60             RAMAddr <= PlayerID_from_Auth;
61             State <= FETCHRAM;
62           end
63
64           else begin
65             State <= WAITFORSORE;
66           end
67         end
68       endcase
69     end
70   end
```

```

64 else begin
65     State <= WAITFORSORE;
66 end
67 end
68
69 FETCHRAM: begin
70     R_W_en <= LOW;
71     State <= RAMWAIT1;
72 end
73
74 RAMWAIT1: begin
75     R_W_en <= LOW;
76     State <= RAMWAIT2;
77 end
78
79 RAMWAIT2: begin
80     R_W_en <= LOW;
81     State <= CATCHRAM;
82 end
83
84 CATCHRAM: begin
85     R_W_en <= LOW;
86     RAMdata_reg <= data_fromRAM;
87     MyBest_reg <= data_fromRAM[7:0]; // ZM: Bug 3 fix -- always refresh MyBest_reg from the current player's RAM slot. Preve
88     State <= COMPARE;
89 end
90
91 COMPARE: begin
92     State <= WRITETORAM;
93
94     if (RAMdata_reg [7:0] < LatestScore_reg) begin
95         MyBest_reg <= LatestScore_reg;
96         data_toRAM <= {NumberID_from_Auth, LatestScore_reg}; // ZM: was '=' (blocking), now '<=' to match the rest of the FSM
97         R_W_en <= HIGH;
98         RAMAddr <= PlayerID_from_Auth;
99     end
100 end
101
102 WRITETORAM: begin
103     R_W_en <= LOW;
104     State <= SCOREOUTPUT;
105 end
106
107 SCOREOUTPUT: begin
108     R_W_en <= LOW;
109     State <= WAITFORSORE;
110
111     // Update global best AND output snapshot in one cycle
112     if (LatestScore_reg > GlobalBest_reg) begin
113         GlobalBest_reg <= LatestScore_reg;
114         BestPlayer_reg <= NumberID_from_Auth;
115         GlobalBest_to_DispCnt <= {NumberID_from_Auth, LatestScore_reg};
116     end
117     else begin
118         GlobalBest_to_DispCnt <= {BestPlayer_reg, GlobalBest_reg};
119     end
120
121     MyBest_to_DispCnt <= {NumberID_from_Auth, MyBest_reg};
122 end
123
124 default: begin
125     R_W_en <= HIGH;
126     RAMAddr <= RAMAddr + 1'b1;
127     data_toRAM <= 24'h000000; // ZM: was 0x111111 -- match RAMINIT
128

```

```

119 end
120
121     MyBest_to_DispCnt <= {NumberID_from_Auth, MyBest_reg};
122 end
123
124 default: begin
125     R_W_en <= HIGH;
126     RAMAddr <= RAMAddr + 1'b1;
127     data_toRAM <= 24'h000000; // ZM: was 0x111111 -- match RAMINIT
128
129     if (RAMAddr == 5'd31) begin
130         State <= WAITFORSORE;
131     end
132 end
133
134 endcase
135 end
136
137 end

```

scoreRAM_top

```

1  scoreRAM_top.v
2  module scoreRAM_top(pulse_WRAM_from_GameCtrl, pulse_sctrl_RAM_from_GameCtrl, PlayerID_from_Auth, NumberID_from_Auth,
3  LatestScore, clk, reset, MyBest_to_DispCnt, GlobalBest_to_DispCnt);
4
5  input pulse_WRAM_from_GameCtrl, pulse_sctrl_RAM_from_GameCtrl, clk, reset;
6  input [4:0] PlayerID_from_Auth;
7  input [15:0] NumberID_from_Auth;
8  input [7:0] LatestScore;
9
10 // ZM: was [7:0] -- truncated the top 16 bits (NumberID) off the concatenated
11 // (NumberID_from_Auth, MyBest_reg) signal that scoreRAM_Ctrl outputs.
12 // Widened to [23:0] to carry the full 24-bit packed value (16-bit ID + 8-bit score).
13 output [23:0] MyBest_to_DispCnt, GlobalBest_to_DispCnt;
14
15 wire [23:0] data_fromRAM, data_toRAM;
16 wire R_W_en;
17 wire [4:0] RAMAddr;
18
19 scoreRAM_Ctrl scoreRAM_Ctrl_top(pulse_WRAM_from_GameCtrl, pulse_sctrl_RAM_from_GameCtrl, PlayerID_from_Auth,
20 NumberID_from_Auth, LatestScore, data_fromRAM, clk, reset, R_W_en, RAMAddr, data_toRAM, MyBest_to_DispCnt, GlobalBest_to_DispCnt);
21
22 score_RAM score_RAM_top(RAMAddr, clk, data_toRAM, R_W_en, data_fromRAM);
23
24
25 endmodule
26

```

twoDigitTimer

```

16 twoDigitTimer.v
17 module twoDigitTimer(clk, rst, time_reconfig, timer_enable,
18 ones_count, tens_count, timer_done);
19 input clk, rst, time_reconfig, timer_enable;
20 output wire [3:0] ones_count;
21 output wire [3:0] tens_count;
22 output wire timer_done;
23
24 wire ones_borrowUp, ones_noBorrowDwn;
25 wire tens_borrowUp, tens_noBorrowDwn;
26 wire timeout;
27
28 //oneSecTimer(clk, rst, enable, timeout);
29 oneSecTimerLFSR timer(clk, rst, timer_enable, timeout);
30
31 // digitTimer(rst, enable, reconfig, borrowDwn, noBorrowUp, borrowUp, noBorrowDwn, count);
32 // ones digit: pulse drives borrowDwn, tens tells it if empty
33 digitTimer onesTimer(rst, timer_enable, time_reconfig, timeout, tens_noBorrowDwn,
34 ones_borrowUp, ones_noBorrowDwn, ones_count);
35
36 // tens digit: ones borrowUp drives borrowDwn, nothing above
37 digitTimer tensTimer(rst, timer_enable, time_reconfig, ones_borrowUp, 1'b1,
38 tens_borrowUp, tens_noBorrowDwn, tens_count);
39
40 assign timer_done = (ones_count == 4'b0000) && (tens_count == 4'b0000) && timer_enable;
41
42 endmodule

```